Project Notes 1 & 2

Problem Statement

An aviation company that provides domestic as well as international trips to the customers now wants to apply a targeted approach instead of reaching out to each of the customers. This time they want to do it digitally instead of tele-calling. Hence they have collaborated with a social networking platform, so they can learn the digital and social behavior of the customers and provide the digital advertisement on the user page of the targeted customers who have a high propensity to take up the product. Propensity of buying tickets is different for different login devices. Hence, you have to create 2 models separately for Laptop and Mobile.

NOTE: Anything which is not a laptop can be considered as mobile phone usage.

NOTE: The advertisements on the digital platform are bit expensive, hence you need to be very accurate while creating the models.

Importing Libraries and Checking the Data :-

In [1]:
```
#Importing basic Libraries

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

#to ignore warnings
import warnings
warnings.filterwarnings('ignore')
```

In [2]:
```
#Loading the dataset

data = pd.read_csv(r'D:\SHUBHANK !\GL\My_Capstone_Project\Social Media Data for DSBA.cs
```

Exploring the data :-

In [3]:
```
#Reading the head of the dataset

data.head().T
```

Out[3]:

	0	1	2	3	4
UserID	1000001	1000002	1000003	1000004	1000005
Taken_product	Yes	No	Yes	No	No
Yearly_avg_view_on_travel_page	307.0	367.0	277.0	247.0	202.0

	0	1	2	3	4
preferred_device	iOS and Android	iOS	iOS and Android	iOS	iOS and Android
total_likes_on_outstation_checkin_given	38570.0	9765.0	48055.0	48720.0	20685.0
yearly_avg_Outstation_checkins	1	1	1	1	1
member_in_family	2	1	2	4	1
preferred_location_type	Financial	Financial	Other	Financial	Medical
Yearly_avg_comment_on_travel_page	94.0	61.0	92.0	56.0	40.0
total_likes_on_outofstation_checkin_received	5993	5130	2090	2909	3468
week_since_last_outstation_checkin	8	1	6	1	9
following_company_page	Yes	No	Yes	Yes	No
montly_avg_comment_on_company_page	11	23	15	11	12
working_flag	No	Yes	No	No	No
travelling_network_rating	1	4	2	3	4
Adult_flag	0	1	0	0	1
Daily_Avg_mins_spend_on_traveling_page	8	10	7	8	6

In [4]:
```
#Printing the shape and size of the dataset

print('Total Columns: ',data.shape[1])
print('Total Rows: ',data.shape[0])


#Size of the dataset
print('\nSize of the data:',data.size)
```

```
Total Columns:  17
Total Rows:  11760

Size of the data: 199920
```

In [5]:
```
#Let see all the names of the columns in the dataset

print('Names of the columns:-\n\n',data.columns)
```

```
Names of the columns:-

 Index(['UserID', 'Taken_product', 'Yearly_avg_view_on_travel_page',
       'preferred_device', 'total_likes_on_outstation_checkin_given',
       'yearly_avg_Outstation_checkins', 'member_in_family',
       'preferred_location_type', 'Yearly_avg_comment_on_travel_page',
       'total_likes_on_outofstation_checkin_received',
       'week_since_last_outstation_checkin', 'following_company_page',
       'montly_avg_comment_on_company_page', 'working_flag',
       'travelling_network_rating', 'Adult_flag',
       'Daily_Avg_mins_spend_on_traveling_page'],
      dtype='object')
```

Renaming few of the columns for better readability and understanding:-

Replacing few long words in short abbreviations, like:

1. Yearly_avg as YA
2. Daily_avg as DA
3. Preffered as Pref
4. Total_likes as TL
5. Monthly_avg as MA
6. Outstation or out of station are same, so replacing it with OS
7. Travel_page and Company_page are same, so replacing it with CP

In [6]:
```
data.rename({'Yearly_avg_view_on_travel_page': 'YA_view_on_CP',
             'preferred_device': 'pref_device',
            'total_likes_on_outstation_checkin_given': 'TL_done_on_OS_checkin',
            'yearly_avg_Outstation_checkins' : 'YA_OS_checkin',
            'member_in_family' : 'members_in_fam',
            'preferred_location_type' : 'pref_loc_type',
            'Yearly_avg_comment_on_travel_page' : 'YA_comment_on_CP',
            'total_likes_on_outofstation_checkin_received' : 'TL_rec_on_OS_checkin',
            'week_since_last_outstation_checkin' : 'week_since_last_OS_checkin',
            'following_company_page' : 'following_CP',
            'montly_avg_comment_on_company_page':'MA_comment_on_CP',
            'Daily_Avg_mins_spend_on_traveling_page' : 'DA_mins_spend_on_CP'}, axis=1,
```

In [7]:
```
#Now again checking columns and shape

print(data.shape)
print('\n---------------')
new_column_names = data.columns
print(new_column_names)
```

```
(11760, 17)

---------------
Index(['UserID', 'Taken_product', 'YA_view_on_CP', 'pref_device',
       'TL_done_on_OS_checkin', 'YA_OS_checkin', 'members_in_fam',
       'pref_loc_type', 'YA_comment_on_CP', 'TL_rec_on_OS_checkin',
       'week_since_last_OS_checkin', 'following_CP', 'MA_comment_on_CP',
       'working_flag', 'travelling_network_rating', 'Adult_flag',
       'DA_mins_spend_on_CP'],
      dtype='object')
```

In [8]:
```
#Let's look at the basic information about the dataset

data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11760 entries, 0 to 11759
Data columns (total 17 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   UserID                     11760 non-null  int64
```

```
 1   Taken_product               11760 non-null  object
 2   YA_view_on_CP               11179 non-null  float64
 3   pref_device                 11707 non-null  object
 4   TL_done_on_OS_checkin       11379 non-null  float64
 5   YA_OS_checkin               11685 non-null  object
 6   members_in_fam              11760 non-null  object
 7   pref_loc_type               11729 non-null  object
 8   YA_comment_on_CP            11554 non-null  float64
 9   TL_rec_on_OS_checkin        11760 non-null  int64
 10  week_since_last_OS_checkin  11760 non-null  int64
 11  following_CP                11657 non-null  object
 12  MA_comment_on_CP            11760 non-null  int64
 13  working_flag                11760 non-null  object
 14  travelling_network_rating   11760 non-null  int64
 15  Adult_flag                  11760 non-null  int64
 16  DA_mins_spend_on_CP         11760 non-null  int64
dtypes: float64(3), int64(7), object(7)
memory usage: 1.5+ MB
```

In [9]:

```
#Description of the data and five number summary using describe function

pd.set_option('display.float_format', lambda x: '%.2f' % x) #Keeping the decimal points
data.describe().T
```

Out[9]:

	count	mean	std	min	25%	50%	75
UserID	11760.00	1005880.50	3394.96	1000001.00	1002940.75	1005880.50	1008820.
YA_view_on_CP	11179.00	280.83	68.18	35.00	232.00	271.00	324.
TL_done_on_OS_checkin	11379.00	28170.48	14385.03	3570.00	16380.00	28076.00	40525.
YA_comment_on_CP	11554.00	74.79	24.03	3.00	57.00	75.00	92.
TL_rec_on_OS_checkin	11760.00	6531.70	4706.61	1009.00	2940.75	4948.00	8393.
week_since_last_OS_checkin	11760.00	3.20	2.62	0.00	1.00	3.00	5.
MA_comment_on_CP	11760.00	28.66	48.66	11.00	17.00	22.00	27.
travelling_network_rating	11760.00	2.71	1.08	1.00	2.00	3.00	4.
Adult_flag	11760.00	0.79	0.85	0.00	0.00	1.00	1.
DA_mins_spend_on_CP	11760.00	13.82	9.07	0.00	8.00	12.00	18.

In [10]:

```
#Checking the data type for each variable in the dataset

data.dtypes
```

Out[10]:

```
UserID                      int64
Taken_product              object
YA_view_on_CP             float64
pref_device                object
TL_done_on_OS_checkin     float64
YA_OS_checkin              object
members_in_fam             object
pref_loc_type              object
```

```
YA_comment_on_CP              float64
TL_rec_on_OS_checkin            int64
week_since_last_OS_checkin      int64
following_CP                   object
MA_comment_on_CP                int64
working_flag                   object
travelling_network_rating       int64
Adult_flag                      int64
DA_mins_spend_on_CP             int64
dtype: object
```

In [11]:
```
#Any null or missing values in dataset ?? Let's check

data.isnull().any()
```

Out[11]:
```
UserID                        False
Taken_product                 False
YA_view_on_CP                  True
pref_device                    True
TL_done_on_OS_checkin          True
YA_OS_checkin                  True
members_in_fam                False
pref_loc_type                  True
YA_comment_on_CP               True
TL_rec_on_OS_checkin          False
week_since_last_OS_checkin    False
following_CP                   True
MA_comment_on_CP              False
working_flag                  False
travelling_network_rating     False
Adult_flag                    False
DA_mins_spend_on_CP           False
dtype: bool
```

Boolean value False means there is no missing data in the column and True means there are some missing values in that column.

In [12]:
```
#Yes, there are missing values in the dataset.

#let's check how many missing values are there in those columns

data.isnull().sum()
```

Out[12]:
```
UserID                          0
Taken_product                   0
YA_view_on_CP                 581
pref_device                    53
TL_done_on_OS_checkin         381
YA_OS_checkin                  75
members_in_fam                  0
pref_loc_type                  31
YA_comment_on_CP              206
TL_rec_on_OS_checkin            0
week_since_last_OS_checkin      0
following_CP                  103
MA_comment_on_CP                0
working_flag                    0
travelling_network_rating       0
```

```
Adult_flag                  0
DA_mins_spend_on_CP         0
dtype: int64
```

In [13]:
```
#Total count of the missing values

print('Total missing values:',data.isnull().sum().sum())
```

```
Total missing values: 1430
```

Data Cleaning :-

Dropping 2 columns that are not useful for our analysis :-

In [14]:
```
#First of all dropping the UserID column as it is not so useful for our analysis
#Also we are dropping montly_avg_comment_on_company_page column bceause we already have
#it does not make sense to keep both columns as it is indicating same action

data.drop(['UserID','MA_comment_on_CP'], axis = 1, inplace = True)
```

Treating Missing Values :-

In [15]:
```
#While checking, we've got 1430 total missing values in the dataset, which is around 12
#So we decide not to do any random imputation because it may affect our model

#Lets just drop them all

data.dropna(inplace=True)
```

In [16]:
```
#After dropping, Let's check the null values again

data.isnull().sum()
```

Out[16]:
```
Taken_product               0
YA_view_on_CP               0
pref_device                 0
TL_done_on_OS_checkin       0
YA_OS_checkin               0
members_in_fam              0
pref_loc_type               0
YA_comment_on_CP            0
TL_rec_on_OS_checkin        0
week_since_last_OS_checkin  0
following_CP                0
working_flag                0
travelling_network_rating   0
Adult_flag                  0
DA_mins_spend_on_CP         0
dtype: int64
```

Now all the null values are dropped from the dataset.

In [17]:
```
#Let's check the shape and info again
```

```
print('Total Columns: ',data.shape[1])
print('Total Rows: ',data.shape[0])
print('\n------------')
data.info()
```

```
Total Columns:  15
Total Rows:  10456

------------
<class 'pandas.core.frame.DataFrame'>
Int64Index: 10456 entries, 0 to 11759
Data columns (total 15 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Taken_product               10456 non-null  object
 1   YA_view_on_CP               10456 non-null  float64
 2   pref_device                 10456 non-null  object
 3   TL_done_on_OS_checkin       10456 non-null  float64
 4   YA_OS_checkin               10456 non-null  object
 5   members_in_fam              10456 non-null  object
 6   pref_loc_type               10456 non-null  object
 7   YA_comment_on_CP            10456 non-null  float64
 8   TL_rec_on_OS_checkin        10456 non-null  int64
 9   week_since_last_OS_checkin  10456 non-null  int64
 10  following_CP                10456 non-null  object
 11  working_flag                10456 non-null  object
 12  travelling_network_rating   10456 non-null  int64
 13  Adult_flag                  10456 non-null  int64
 14  DA_mins_spend_on_CP         10456 non-null  int64
dtypes: float64(3), int64(5), object(7)
memory usage: 1.3+ MB
```

Now there are 10456 records and no null/missing values present.

Dropping the rows which has special characters :-

While checking the dataset, we found out that there is a special character in the column 'YA_OS_checkin'. So let's drop the rows with special character.

In [18]:
```
#First checking the row with special character in the column
data[data.YA_OS_checkin.str.contains(r'[@#&$%+-/*]')]

#dropping the complete row
data.drop(data[data.YA_OS_checkin.str.contains(r'[^0-9a-zA-Z]')].index, inplace=True)
```

In [19]:
```
#reprinting the shape now

data.shape
```

Out[19]:
```
(10455, 15)
```

Now we are left with 10455 rows and 15 columns.

Let's do the Univariate Analysis :-

In [20]:
```
#Taken_product (Whether the user has taken their product or not)
```

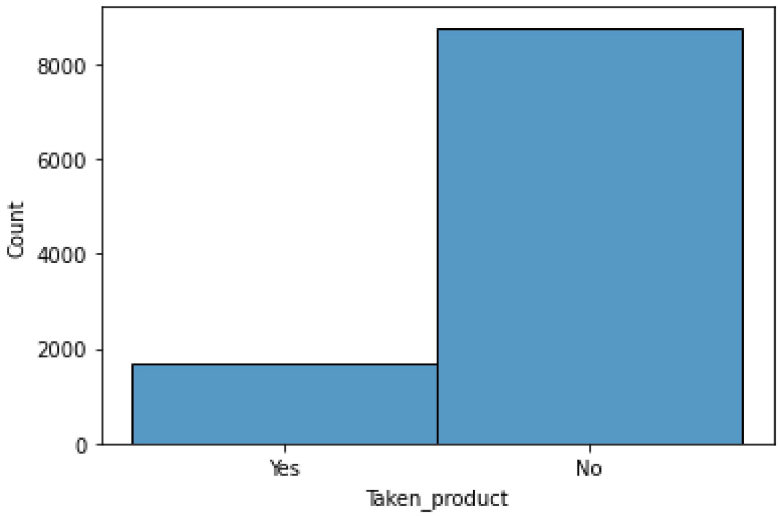
```
print(data.Taken_product.value_counts())
print('\n---------------')
print('Proportion:\n\n',data.Taken_product.value_counts(normalize=True))
```

```
No     8762
Yes    1693
Name: Taken_product, dtype: int64

---------------
Proportion:

 No    0.84
Yes   0.16
Name: Taken_product, dtype: float64
```

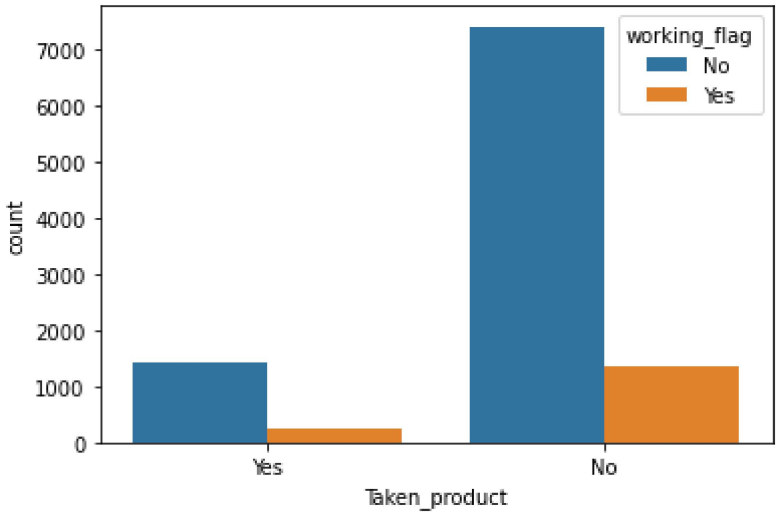
84% people did not take the product. This means that data is not balanced. We will deal with problem later.

In [21]:
```
#Histogram for Taken_product

sns.histplot(data=data, x="Taken_product")
```

Out[21]:
```
<AxesSubplot:xlabel='Taken_product', ylabel='Count'>
```



In [22]:
```
#Countplot for Taken product with other variables to compare the relation

ax = sns.countplot(x="Taken_product", hue="working_flag", data=data)
```



In [23]:
```
#Pref_device (Preferred device of the user through which user do login)

print(data.pref_device.value_counts())
print('\n---------------')
print('Proportion: \n',data.pref_device.value_counts(normalize=True))
```

```
Tab               4172
iOS and Android    3246
Laptop             1108
iOS                 859
Mobile              600
Android             244
Android OS          126
ANDROID              97
Other                 2
Others                1
Name: pref_device, dtype: int64

---------------
Proportion:
 Tab               0.40
iOS and Android   0.31
Laptop            0.11
iOS               0.08
Mobile            0.06
Android           0.02
Android OS        0.01
ANDROID           0.01
Other             0.00
Others            0.00
Name: pref_device, dtype: float64
```

We can see there are so many devices mentioned in the dataset. But as per the information provided in the problem statement that 'Anything which is not a laptop can be considered as mobile phone usage', so we will replace everything with mobile except Laptop.

In [24]:
```
data['pref_device'].replace({'Tab': 'Mobile', 'iOS and Android': 'Mobile', 'iOS': 'Mobi
                             'Android': 'Mobile', 'Android OS': 'Mobile', 'ANDROID': 'Mo
                             'iOS and Android': 'Mobile', 'Other': 'Mobile', 'Others': '
```

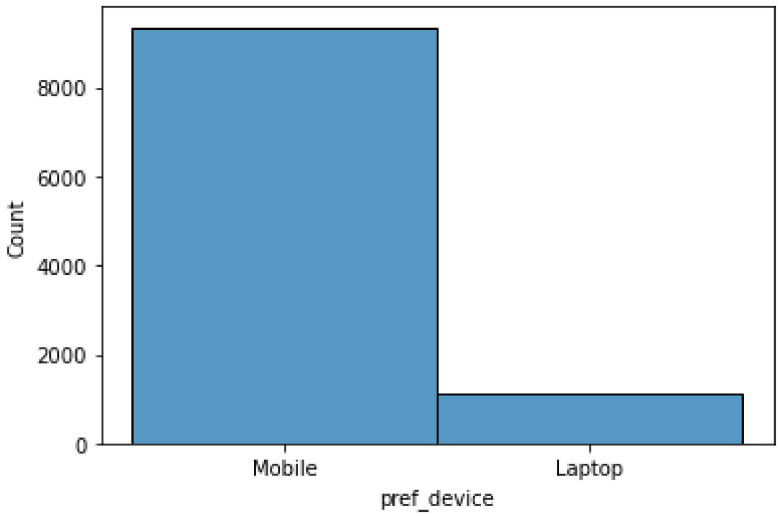
In [25]:
```
#Now again checking pref_device column

print(data.pref_device.value_counts())
print('\n---------------')
print('Proportion: \n',data.pref_device.value_counts(normalize=True))
```

```
Mobile    9347
Laptop    1108
Name: pref_device, dtype: int64

---------------
Proportion:
 Mobile   0.89
Laptop   0.11
Name: pref_device, dtype: float64
```

This indicates that 89% of the people in our data use Mobile as their preferred device.

In [26]:
```
#Histogram for pref_device

sns.histplot(data=data, x="pref_device")
```

Out[26]:
```
<AxesSubplot:xlabel='pref_device', ylabel='Count'>
```



In [27]:
```
#Pref_loc_type  (Preferred location type of the user)

print(data.pref_loc_type.value_counts())
```

```
Beach              2424
Financial          1875
Historical site    1856
Medical            1448
Big Cities          636
Other               581
Trekking            528
Social media        506
Entertainment       383
Hill Stations       108
Tour  Travel         53
Tour and Travel      37
Game                 10
```

```
OTT                    6
Movie                  4
Name: pref_loc_type, dtype: int64
```

We can see that there are various preferred location type available in the data. Let's assign few of them into one category and organize these.

1. We will take Beach, Historical site, Trekking, Tour Travel, Tour and Travel and Hill Stations in one category and will name it 'Adventure and Tourism'.
2. We will be replacing Game, OTT, Movie and Social Media with one already available category 'Entertainment'.

In [28]:
```
data['pref_loc_type'].replace({'Beach': 'Adventure and Tourism','Historical site': 'Adv
                               'Trekking': 'Adventure and Tourism', 'Tour  Travel': 'Ad
                               'Tour and Travel': 'Adventure and Tourism', 'Hill Statio
                               'Game': 'Entertainment', 'OTT': 'Entertainment', 'Movie'
                              'Social media': 'Entertainment'}, inplace=True)
```

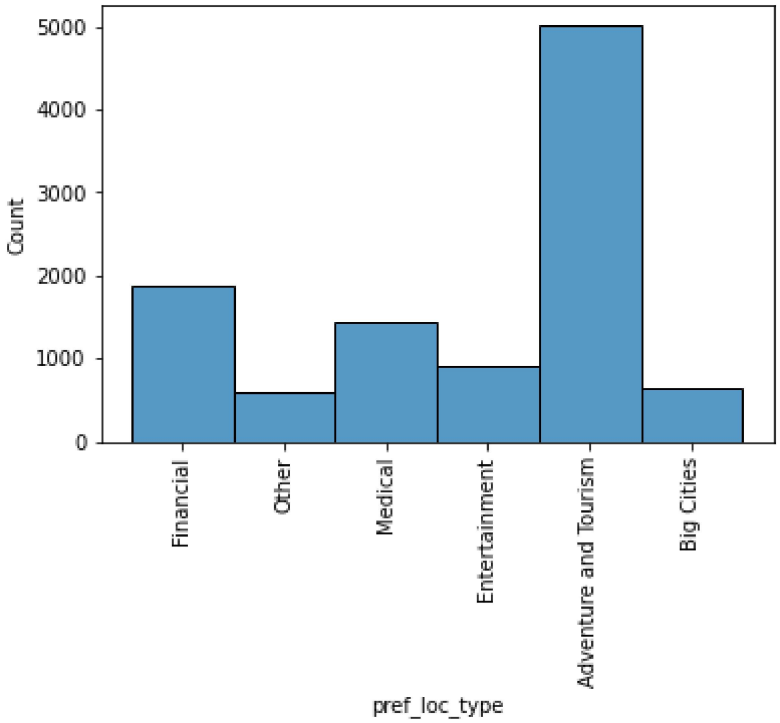
In [29]:
```
#Checking the updated values and checking the proportion for each

print(data.pref_loc_type.value_counts())
print('\n---------------')
print('Proportion: \n',data.pref_loc_type.value_counts(normalize=True))
```

```
Adventure and Tourism    5006
Financial                1875
Medical                  1448
Entertainment             909
Big Cities                636
Other                     581
Name: pref_loc_type, dtype: int64

---------------
Proportion:
 Adventure and Tourism   0.48
Financial               0.18
Medical                 0.14
Entertainment           0.09
Big Cities              0.06
Other                   0.06
Name: pref_loc_type, dtype: float64
```

So after organizing the categories, we found out that 'Adventure and Tourism' is the most preferred location type with 48%. Then 18% of the user has 'Financial' as their preferred location type, because they might travel mostly for business purpose. Then comes the 'Medical' as preferred location, as 14% of the user travel for their medical purposes. Only 9% of the users has 'Entertainment' as their preferred location type.

In [30]:
```
#Histogram for pref_loc_type

plt.xticks(rotation=90)
sns.histplot(data=data, x="pref_loc_type")
```

Out[30]:
```
<AxesSubplot:xlabel='pref_loc_type', ylabel='Count'>
```



In [31]:
```
#Following_CP (whether the user is following the company page or not)

print(data.following_CP.value_counts())
```

```
No     7517
Yes    2923
1        10
0         5
Name: following_CP, dtype: int64
```

Here we need the data in the form of 'Yes' and 'No'. So we need to replace numeric value of 1 to Yes and 0 to No.

In [32]:
```
#replacing 1 and 0 with Yes and No respectively.

data['following_CP'].replace({'1': 'Yes', '0': 'No'}, inplace=True)
```

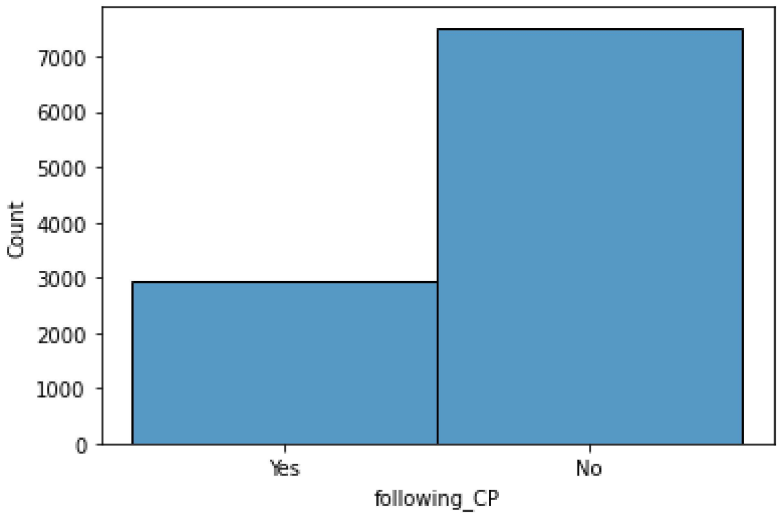
In [33]:
```
#Again checking

print(data.following_CP.value_counts())
print('\n---------------')
print('Proportion: \n',data.following_CP.value_counts(normalize = True))
```

```
No     7522
Yes    2933
Name: following_CP, dtype: int64

---------------
Proportion:
 No    0.72
Yes   0.28
Name: following_CP, dtype: float64
```

We see that only 28% of the users are following the company page and 72% still do not follow.

In [34]:
```
#Histogram for following_CP

sns.histplot(data=data, x="following_CP")
```

Out[34]:
```
<AxesSubplot:xlabel='following_CP', ylabel='Count'>
```



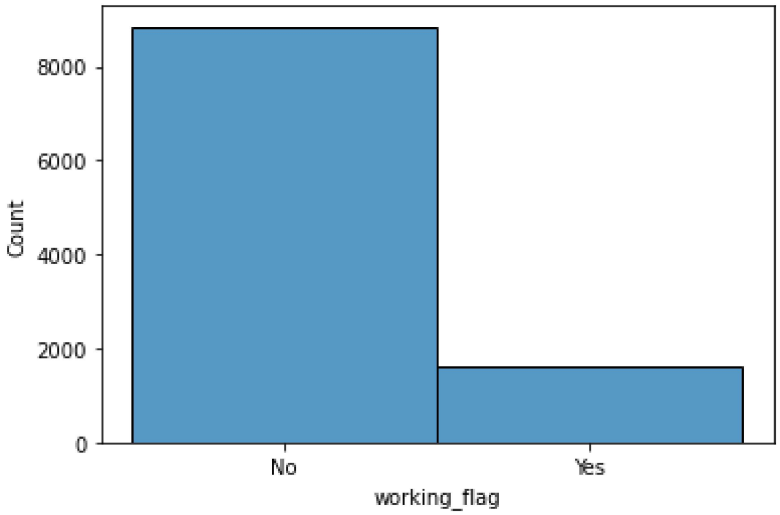
In [35]:
```
#Working_flag (whether the user is working somewhere or not)

print(data.working_flag.value_counts())
print('\n---------------')
print('Proportion: \n',data.working_flag.value_counts(normalize = True))
```

```
No     8846
Yes    1609
Name: working_flag, dtype: int64

---------------
Proportion:
 No    0.85
Yes   0.15
Name: working_flag, dtype: float64
```

85% of the users do not work anywhere. Only 15% users are working.

In [36]:
```
#Histogram for working_flag

sns.histplot(data=data, x="working_flag")
```

Out[36]:
```
<AxesSubplot:xlabel='working_flag', ylabel='Count'>
```



In [37]:
```
#members_in_fam (Number of members in the family of the user)

data.members_in_fam.value_counts()
```

Out[37]:
```
3        4087
4        2843
2        1983
1        1192
5         331
10         10
Three       9
Name: members_in_fam, dtype: int64
```

Instead of numeric 3, it is in alphabet form, let's replace/correct it.

In [38]:
```
data['members_in_fam'].replace({'Three': '3'}, inplace=True)
```

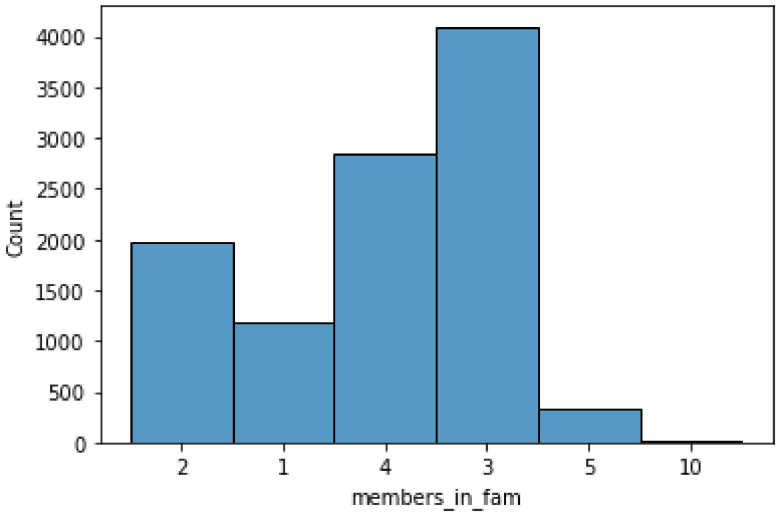
In [39]:
```
data.members_in_fam.value_counts()
```

Out[39]:
```
3     4096
4     2843
2     1983
1     1192
5      331
10      10
Name: members_in_fam, dtype: int64
```

Most of the users have 3 members in their family.

In [40]:
```
#Histogram for members_in_fam

sns.histplot(data=data, x="members_in_fam")
```

Out[40]:
```
<AxesSubplot:xlabel='members_in_fam', ylabel='Count'>
```



In [41]:
```
#Adult_flag (Whether the user is Adult or not)
#it should be either only 0 and 1, where 0 means user is not adult and 1 means user is

data.Adult_flag.value_counts()
```

Out[41]:
```
0    4476
1    4248
2    1114
3     617
Name: Adult_flag, dtype: int64
```

In [42]:
```
#here we will replace 2 and 3 with 1 because it's a categorical variable which must hav
#User can either be adult or not, so it should have values of either yes and no, or 1 a

data['Adult_flag'].replace({2: 1, 3: 1}, inplace=True)
```

In [43]:
```
#Now checking again

data.Adult_flag.value_counts()
```

Out[43]:
```
1    5979
0    4476
Name: Adult_flag, dtype: int64
```

In [44]:
```
data.Adult_flag.value_counts(normalize=True)
```

Out[44]:
```
1   0.57
0   0.43
Name: Adult_flag, dtype: float64
```

Around 57% users are adult.

Correcting the data types of few of the data columns :-

As we have checked that few variables indicating the incorrect data types like:

1. 'YA_view_on_CP' should have integer type instead of float type.
2. 'TL_done_on_OS_checkin' should have integer type instead of float type.

3. 'YA_OS_checkin' should have integer type instead of object type.
4. 'members_in_fam' should have integer type instead of object type.
5. 'YA_comment_on_CP' should have integer type instead of float type.

In [45]:
```
data['YA_view_on_CP'] = data['YA_view_on_CP'].astype(np.int64)
data['TL_done_on_OS_checkin'] = data['TL_done_on_OS_checkin'].astype(np.int64)
data['YA_OS_checkin'] = data['YA_OS_checkin'].astype(np.int64)
data['members_in_fam'] = data['members_in_fam'].astype(np.int64)
data['YA_comment_on_CP'] = data['YA_comment_on_CP'].astype(np.int64)
```

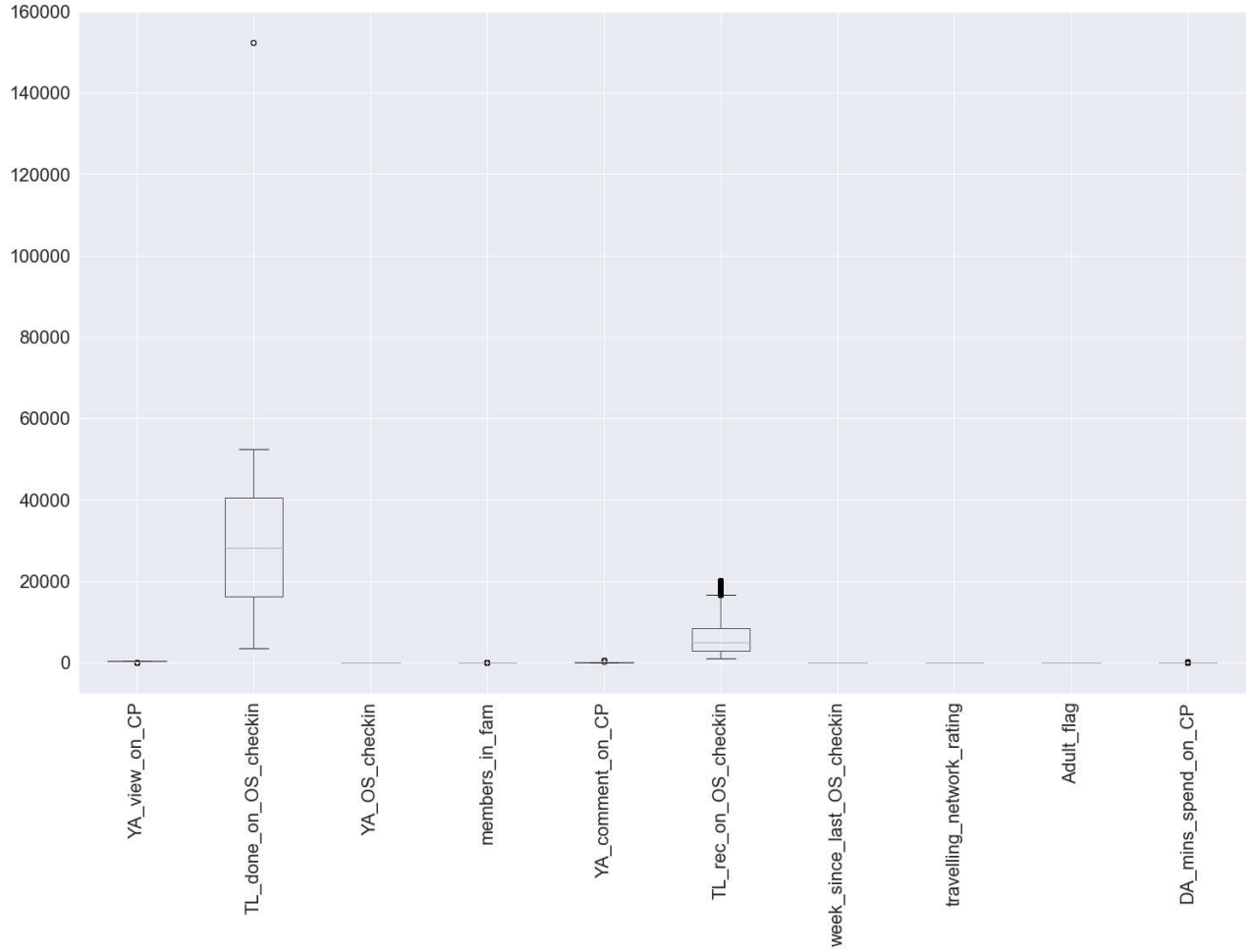
In [46]:
```
#Let's check the dtypes again

data.dtypes
```

Out[46]:
```
Taken_product                object
YA_view_on_CP                 int64
pref_device                  object
TL_done_on_OS_checkin         int64
YA_OS_checkin                 int64
members_in_fam                int64
pref_loc_type                object
YA_comment_on_CP              int64
TL_rec_on_OS_checkin          int64
week_since_last_OS_checkin    int64
following_CP                 object
working_flag                 object
travelling_network_rating     int64
Adult_flag                    int64
DA_mins_spend_on_CP           int64
dtype: object
```

Detecting and Treating the Outliers :-

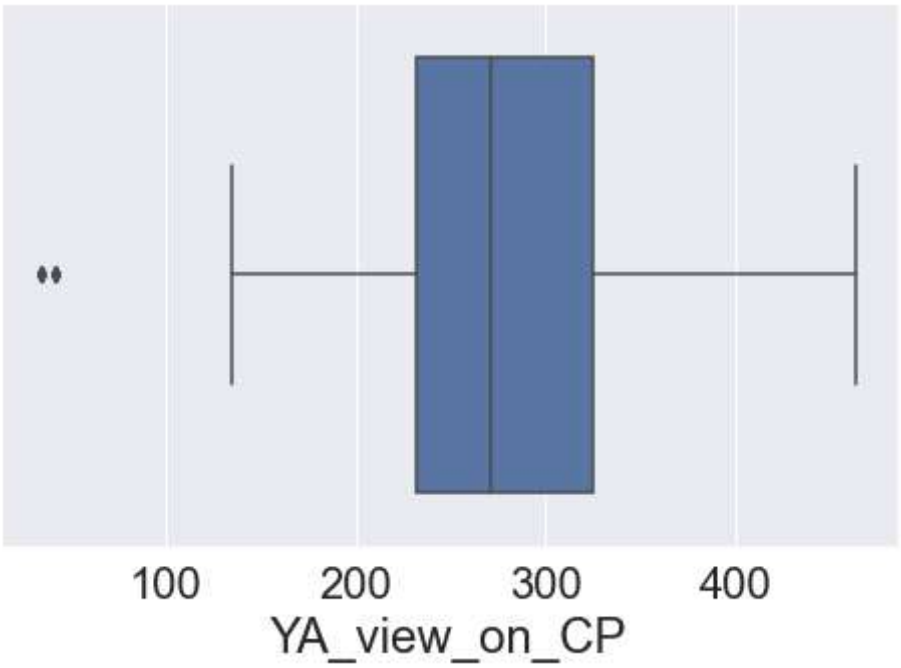
In [47]:
```
#Detecting Outliers by making a box plot for numerical attributes

sns.set(font_scale = 2)
data.boxplot(figsize=(25, 15), rot=90)
```

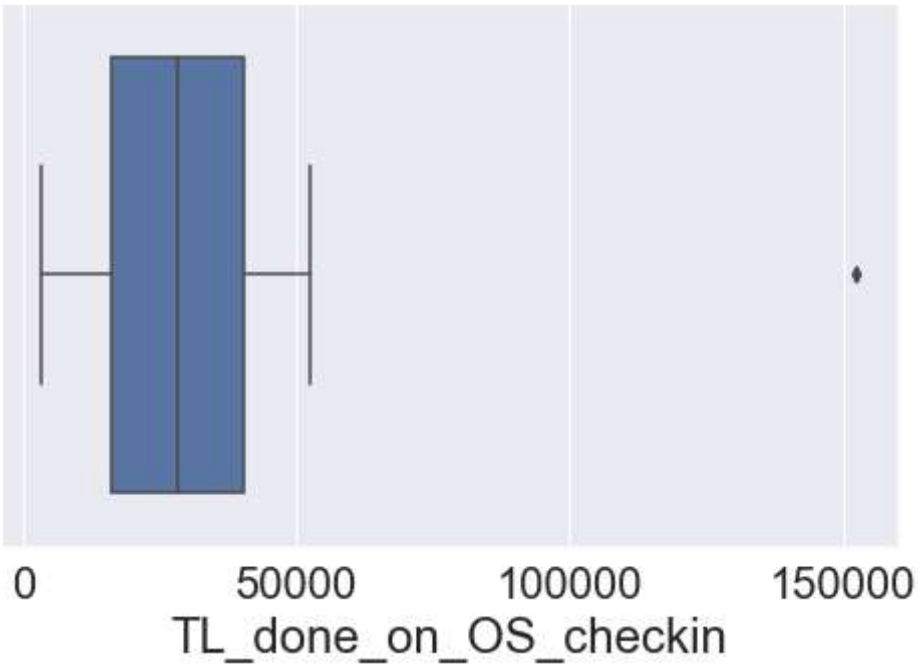
Out[47]:
```
<AxesSubplot:>
```

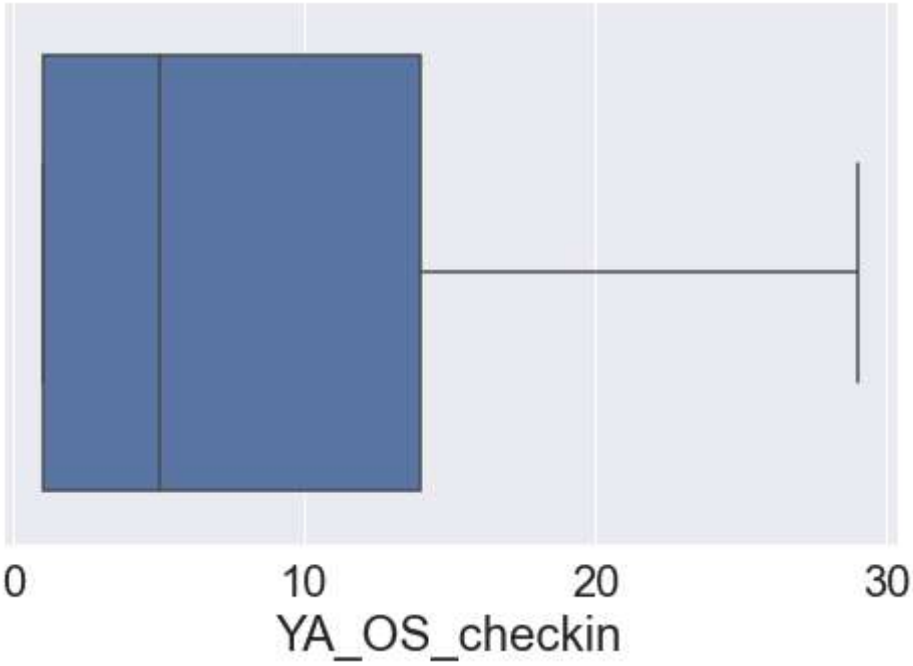


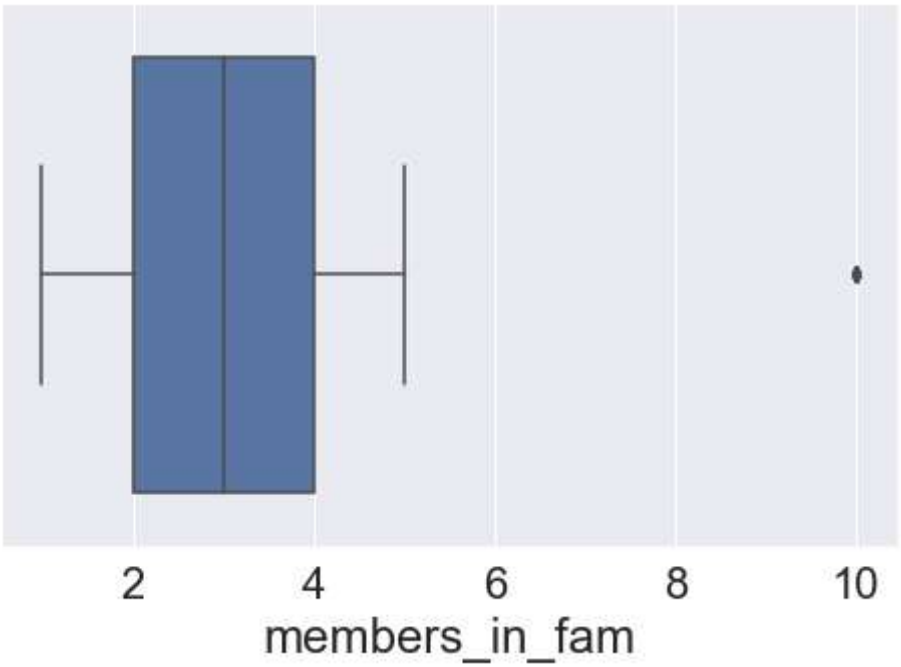
There are few outliers in the data.

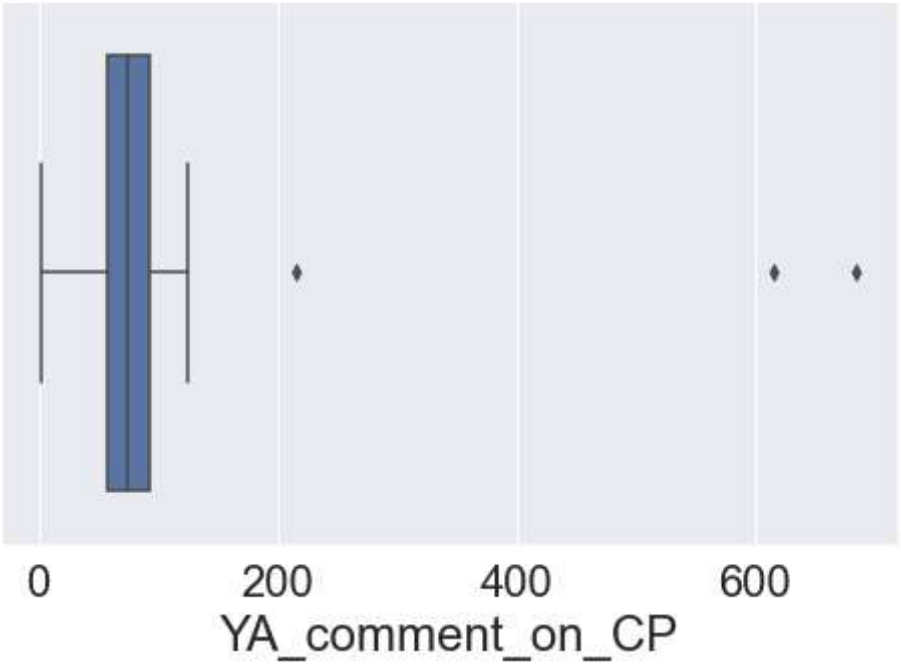
In [48]:
```
#Let's look at them one by one for each numerical variables

num_columns = ['YA_view_on_CP' ,'TL_done_on_OS_checkin', 'YA_OS_checkin', 'members_in_f
               'TL_rec_on_OS_checkin','week_since_last_OS_checkin','travelling_network_
               'DA_mins_spend_on_CP']

for col in num_columns:
    plt.figure(figsize=(8,5))
    sns.boxplot(data[col])
    plt.show()
```

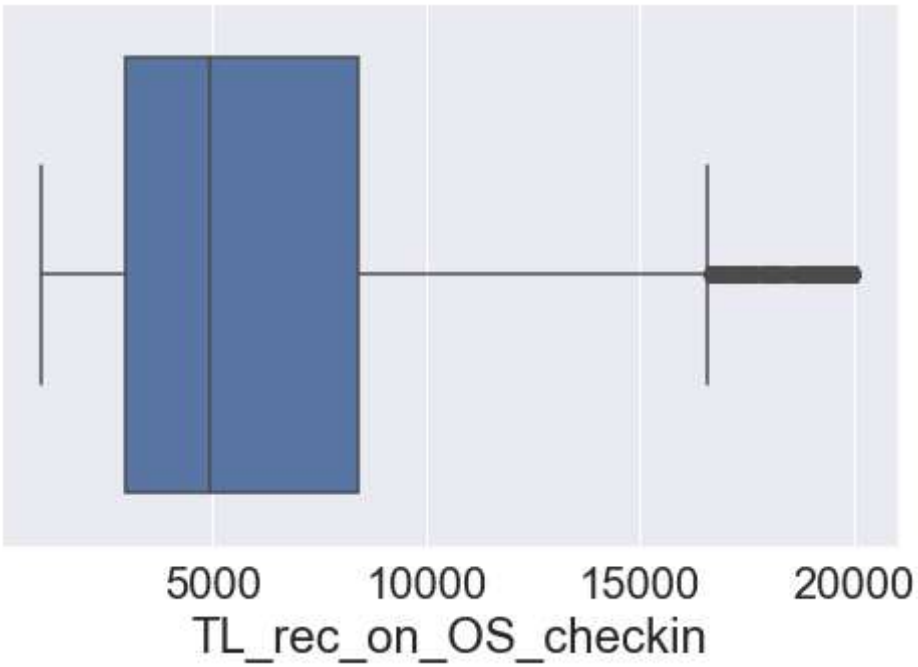

100
200
300
400
YA_view_on_CP

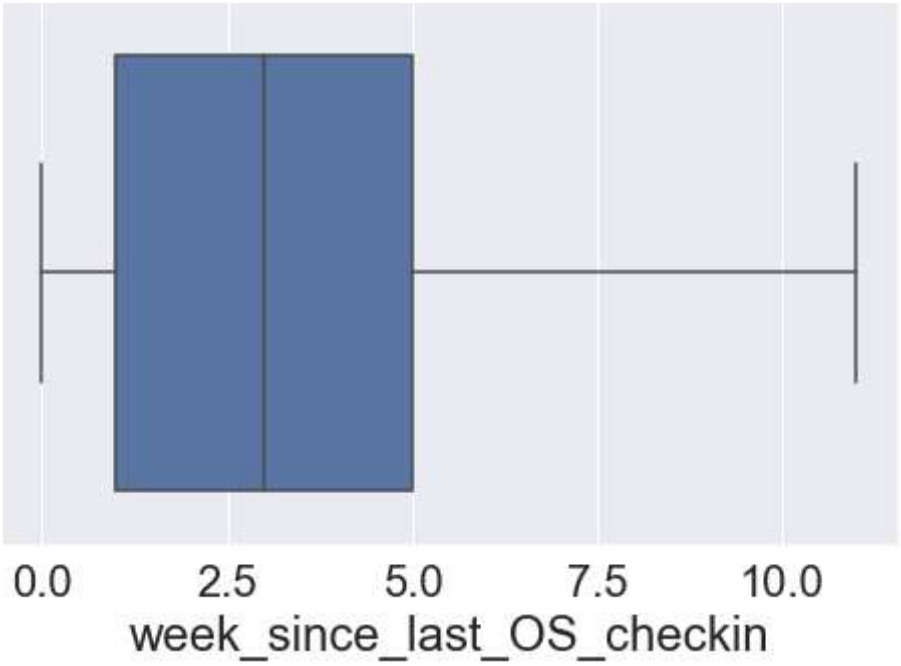

0
50000
100000
150000
TL_done_on_OS_checkin

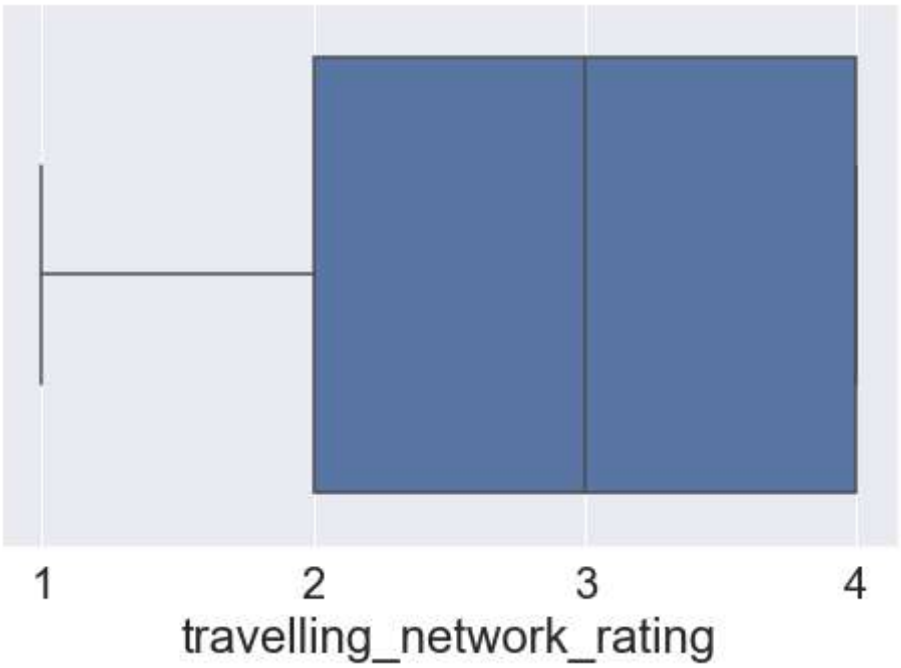

0
10
20
30
YA_OS_checkin

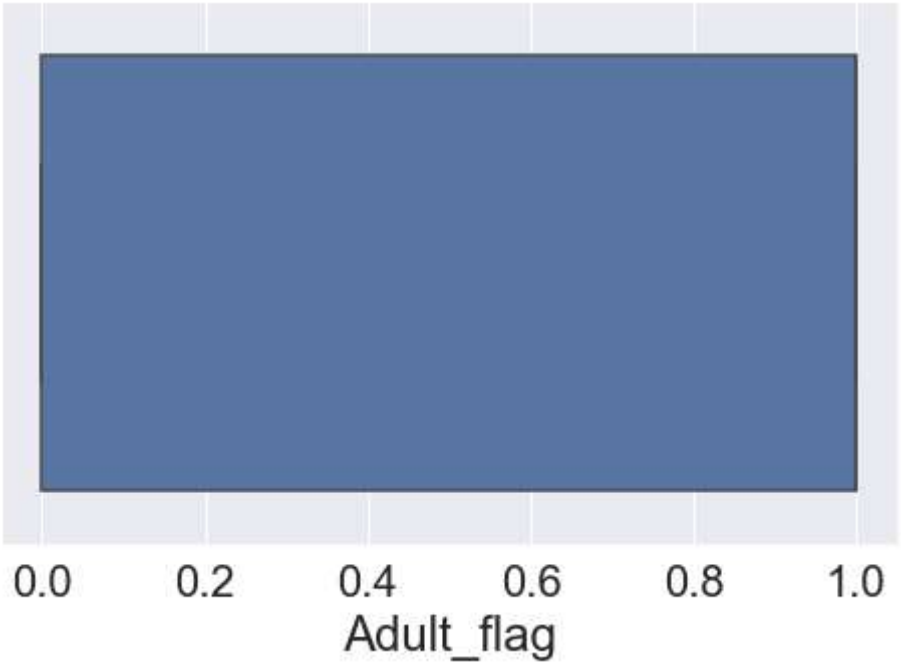

2
4
6
8
10
members_in_fam

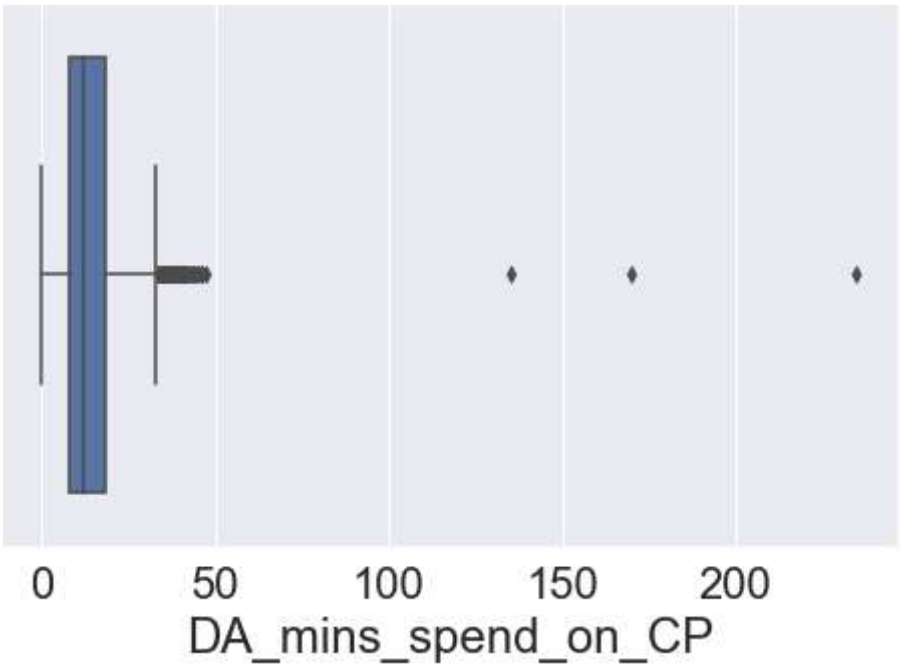

0
200
400
600
YA_comment_on_CP


5000
10000
15000
20000
TL_rec_on_OS_checkin


0.0
2.5
5.0
7.5
10.0
week_since_last_OS_checkin


1
2
3
4
travelling_network_rating





Treating Outliers :-

Treating the outliers by using Inter Quartile Range (IQR) method.This technique uses the IQR scores to remove outliers. The rule of thumb is that anything not in the range of (Q1 - 1.5 IQR) and (Q3 + 1.5 IQR) is an outlier, and can be removed.

In [49]:
```
#Calculating Q1 and Q3 values and assigning the upper limit (UL) and lower limit (LL)

Q1 = data[num_columns].quantile(0.25)
Q3 = data[num_columns].quantile(0.75)
IQR = Q3 - Q1
UL = Q3 + 1.5*IQR
LL = Q1 - 1.5*IQR
```

In [50]:
```
#Checking the number of outliers present in each column

pd.options.display.max_rows = None
((data[num_columns] > UL) | (data[num_columns] < LL)).sum()
```

Out[50]:
```
YA_view_on_CP                  8
TL_done_on_OS_checkin          2
YA_OS_checkin                  0
members_in_fam                10
YA_comment_on_CP               3
TL_rec_on_OS_checkin         818
week_since_last_OS_checkin     0
travelling_network_rating      0
Adult_flag                     0
DA_mins_spend_on_CP          315
dtype: int64
```

In [51]:
```
#Total sum of outliers

print('Total number of outliers in the dataset:',((data[num_columns] > UL) | (data[num_
```

```
Total number of outliers in the dataset: 1156
```

It is almost around 10%, we will remove those from our dataset

We will be defining a funtion named treat_outliers using that function we will remove those outliers.

In [52]:
```
def treat_outliers(col):
    sorted(col)
    Q1,Q3=np.percentile(col,[25,75])
    IQR=Q3-Q1
    lower_range= Q1-(1.5 * IQR)
    upper_range= Q3+(1.5 * IQR)
    return lower_range, upper_range

for column in data[num_columns].columns:
    lr,ur=treat_outliers(data[column])
    data[column]=np.where(data[column]>ur,ur,data[column])
    data[column]=np.where(data[column]<lr,lr,data[column])
```

In [53]:
```
print('Lower Range:',lr)
print('Upper Range:',ur)
```

```
Lower Range: -7.0
Upper Range: 33.0
```

In [54]:
```
#Now checking again.
((data[num_columns] > UL) | (data[num_columns] < LL)).sum()
```

Out[54]:
```
YA_view_on_CP                  0
TL_done_on_OS_checkin          0
YA_OS_checkin                  0
members_in_fam                 0
YA_comment_on_CP               0
TL_rec_on_OS_checkin           0
week_since_last_OS_checkin     0
```

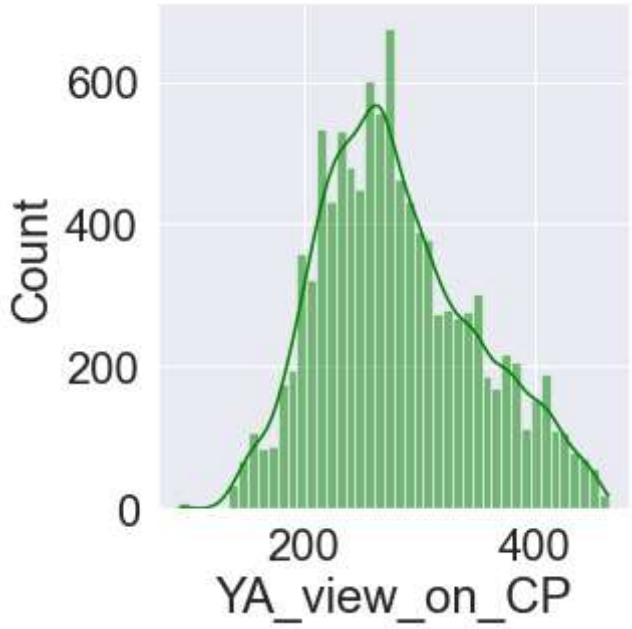
```
travelling_network_rating     0
Adult_flag                    0
DA_mins_spend_on_CP           0
dtype: int64
```

Distribution Plot : For all numerical columns :-

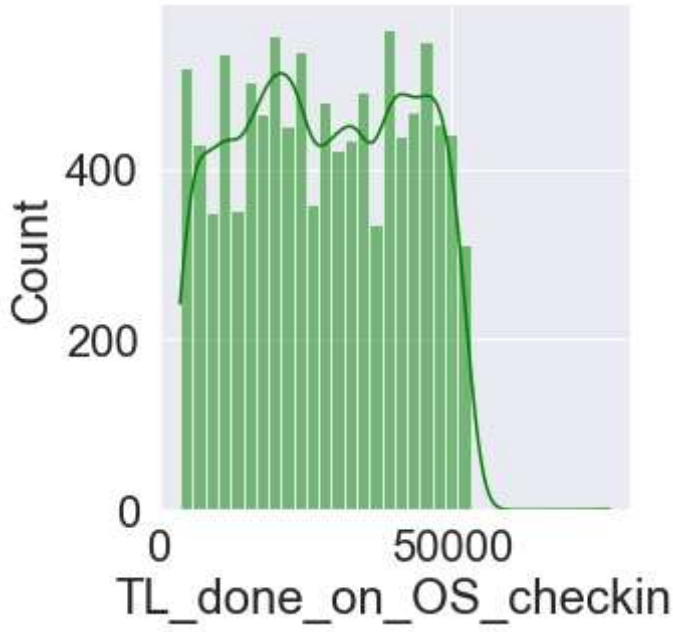
In [55]:

```
for x in num_columns:
    plt.figure(figsize=(10,8))
    sns.displot(data[x], kde=True, color='green')
    plt.show()
```

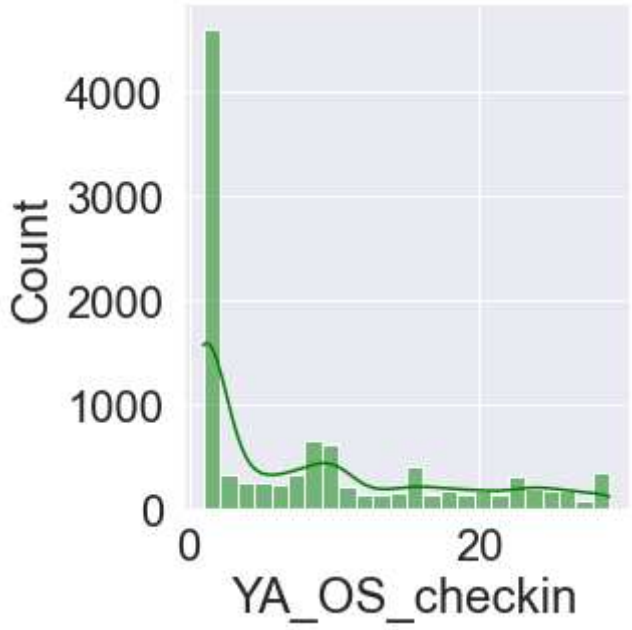
```
<Figure size 720x576 with 0 Axes>
```



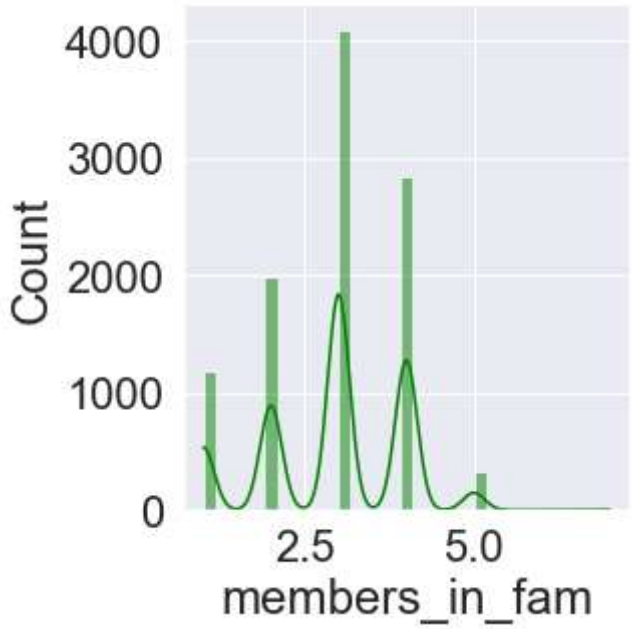

```
<Figure size 720x576 with 0 Axes>
```



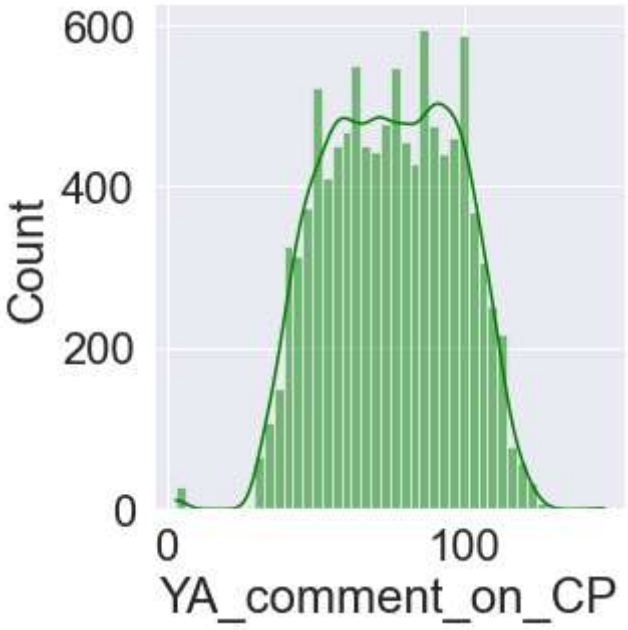

```
<Figure size 720x576 with 0 Axes>
```



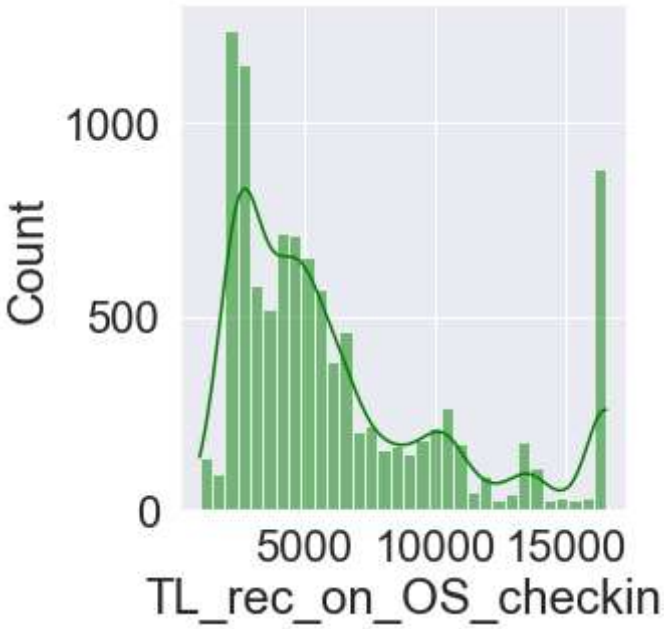

```
<Figure size 720x576 with 0 Axes>
```



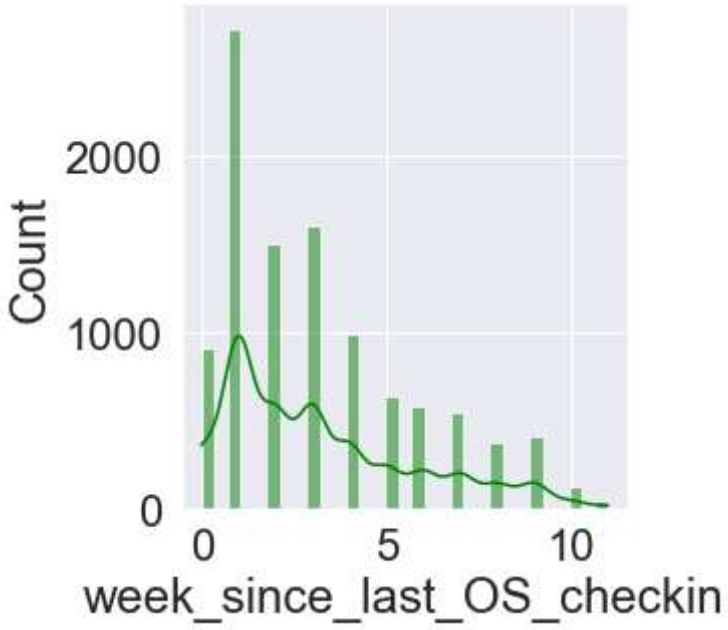

```
<Figure size 720x576 with 0 Axes>
```



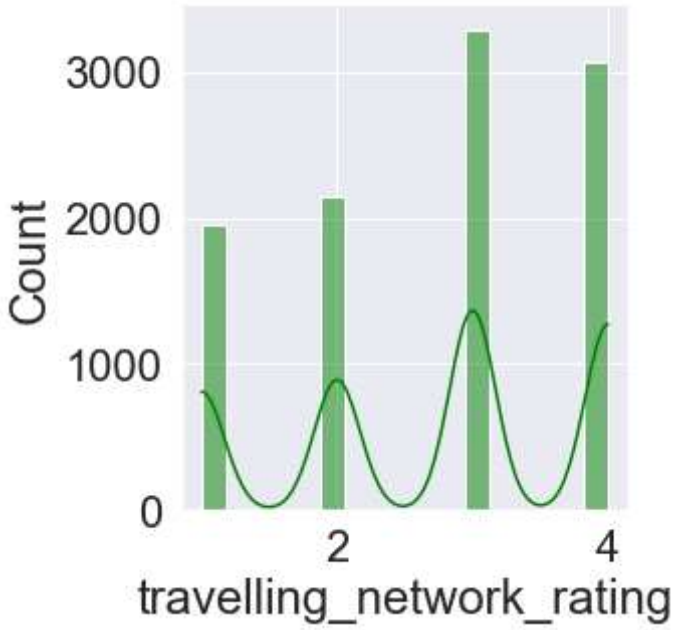

```
<Figure size 720x576 with 0 Axes>
```



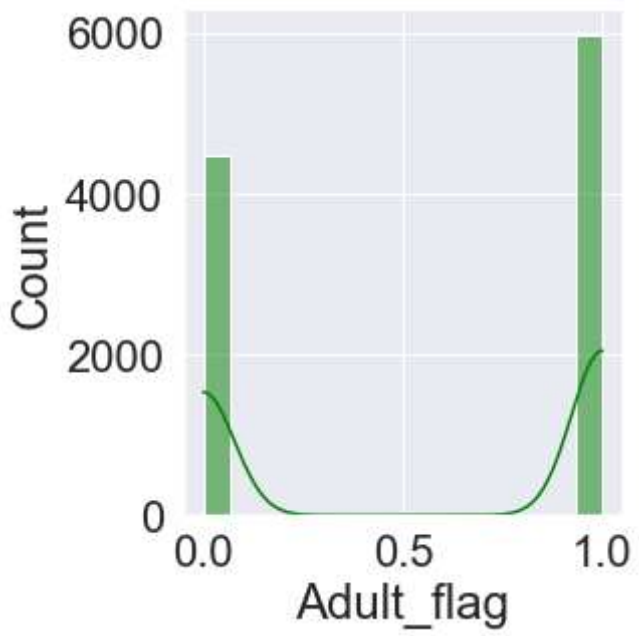

```
<Figure size 720x576 with 0 Axes>
```



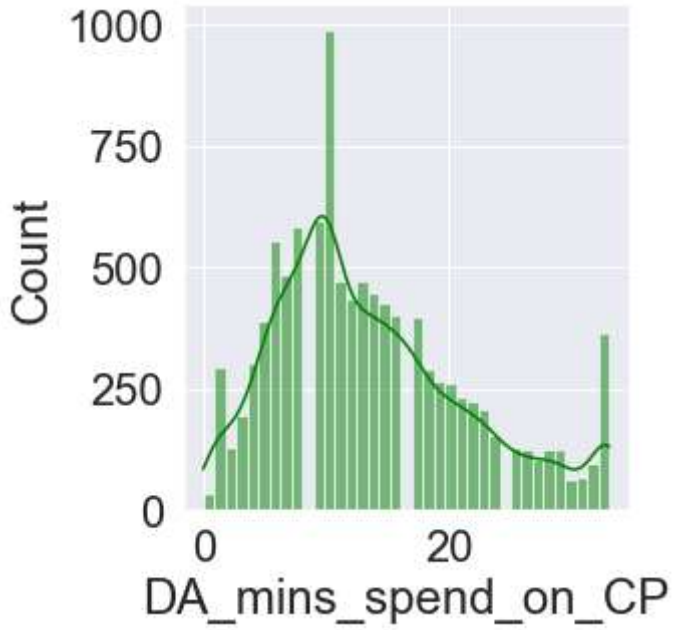

```
<Figure size 720x576 with 0 Axes>
```




```
<Figure size 720x576 with 0 Axes>
```




```
<Figure size 720x576 with 0 Axes>
```



Bivariate Analysis :-

Correlation Matrix :-

In [56]:
```
#Now let see the correlation between the variables using correlation matrics

corr = data.corr()

corr
```

Out[56]:

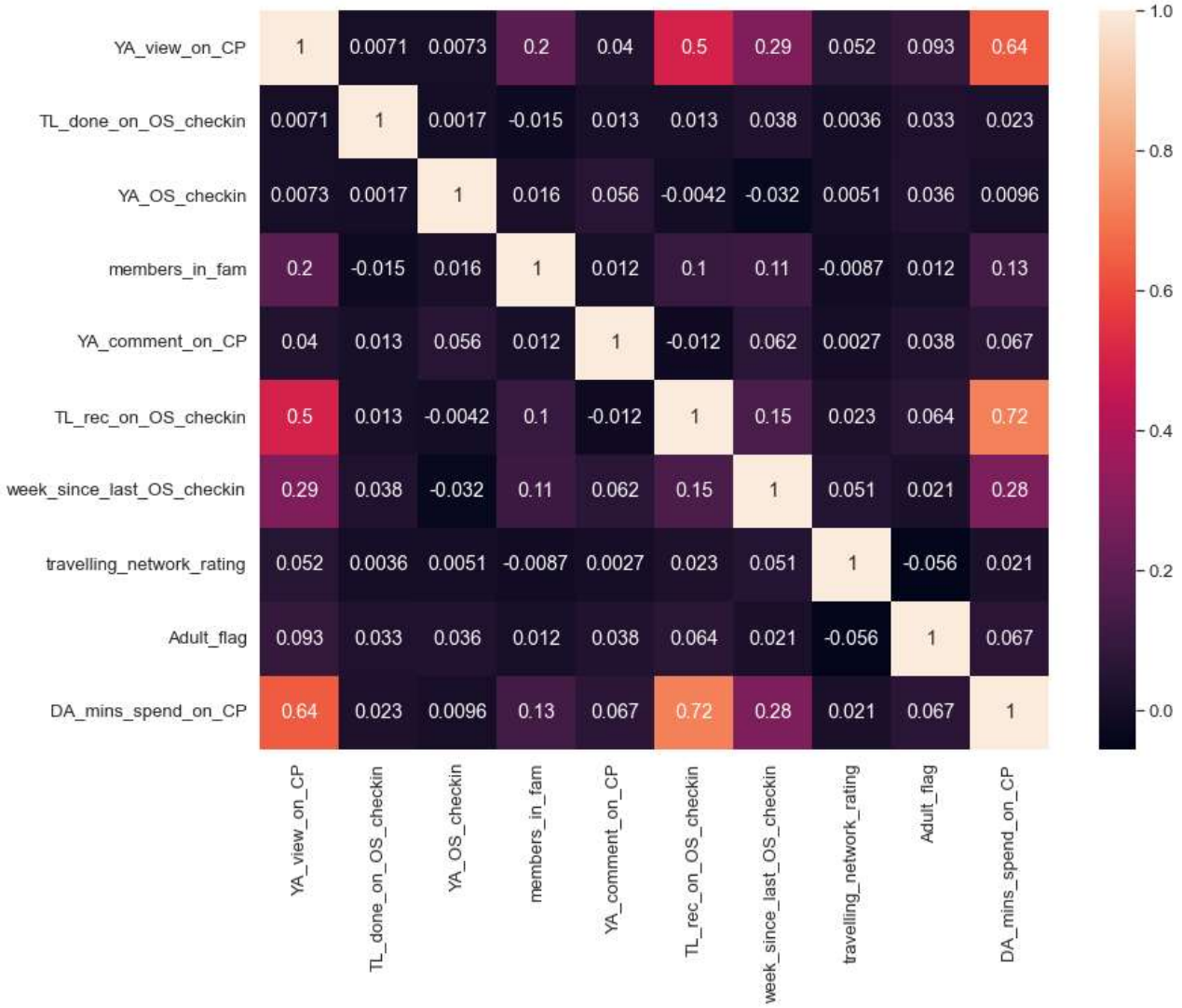
	YA_view_on_CP	TL_done_on_OS_checkin	YA_OS_checkin	members_in_fam	YA
YA_view_on_CP	1.00	0.01	0.01	0.20	
TL_done_on_OS_checkin	0.01	1.00	0.00	-0.01	

	YA_view_on_CP	TL_done_on_OS_checkin	YA_OS_checkin	members_in_fam	YA
YA_OS_checkin	0.01	0.00	1.00	0.02	
members_in_fam	0.20	-0.01	0.02	1.00	
YA_comment_on_CP	0.04	0.01	0.06	0.01	
TL_rec_on_OS_checkin	0.50	0.01	-0.00	0.10	
week_since_last_OS_checkin	0.29	0.04	-0.03	0.11	
travelling_network_rating	0.05	0.00	0.01	-0.01	
Adult_flag	0.09	0.03	0.04	0.01	
DA_mins_spend_on_CP	0.64	0.02	0.01	0.13	

Heatmap :-

In [57]:
```
#Now we will plot the heatmap to display the correlation between the variables:

sns.set(font_scale=1.2)
fig, ax = plt.subplots(figsize=(13,10))
sns.heatmap(corr,annot=True)
```

Out[57]: <AxesSubplot:>



So here in the above heatmap we can see that the light colours shows the highest correlation between 2 variables.

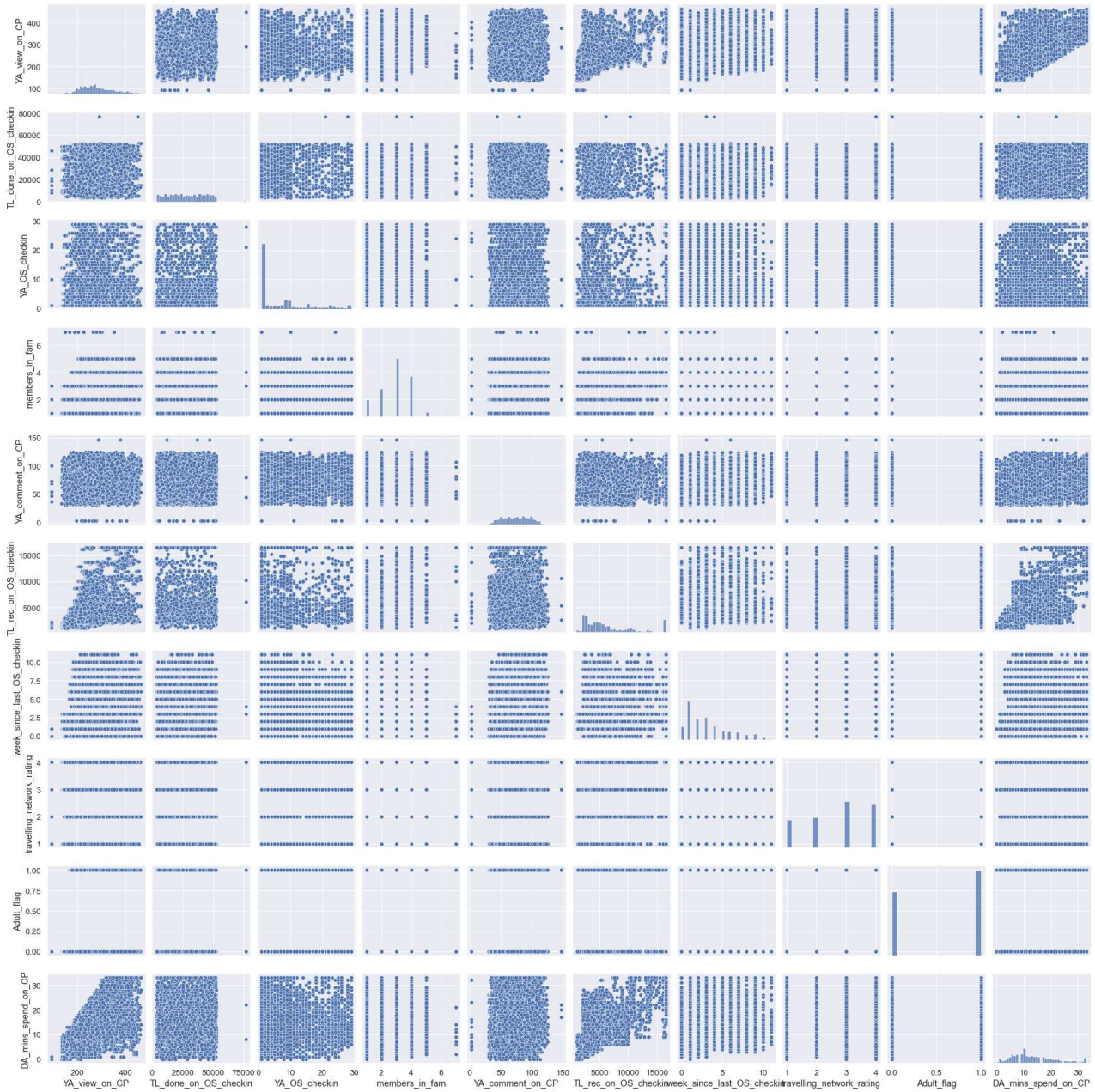
Highest correlated variables are :-

1. 'TL_rec_on_OS_checkin' and 'DA_mins_spend_on_CP' (0.72)
2. 'YA_view_on_CP' and 'DA_mins_spend_on_CP' (0.64)
3. 'TL_rec_on_OS_checkin' and 'YA_view_on_CP' (0.5)

PairPlot :-

In [58]:
```
sns.set(font_scale=1.2)
sns.pairplot(data)
```

Out[58]:
```
<seaborn.axisgrid.PairGrid at 0x23646cc4550>
```



END

Capstone Project: Project Notes - 2

The second set of project notes must include the model building aspect and be aligned to the analytical business problem the Capstone Project addresses. It should also include the rationale behind using/building one particular model (if applicable). The notes should be brief and succinct containing the depiction of the results for the models built aligned with the objective of the project.

1. Model building and interpretation.

1. Build various models (You can choose to build models for either or all of descriptive, predictive or prescriptive purposes).

2. Test your predictive model against the test set using various appropriate performance metrics c.Interpretation of the model(s).

Importing the required libraries :-

In [59]:
```
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import GridSearchCV
from sklearn import metrics
from sklearn.metrics import roc_auc_score,roc_curve,classification_report,confusion_mat
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
```

In [60]:
```
data.head().T
```

Out[60]:

	0	1	2	3	4
Taken_product	Yes	No	Yes	No	No
YA_view_on_CP	307.00	367.00	277.00	247.00	202.00
pref_device	Mobile	Mobile	Mobile	Mobile	Mobile
TL_done_on_OS_checkin	38570.00	9765.00	48055.00	48720.00	20685.00
YA_OS_checkin	1.00	1.00	1.00	1.00	1.00
members_in_fam	2.00	1.00	2.00	4.00	1.00
pref_loc_type	Financial	Financial	Other	Financial	Medical
YA_comment_on_CP	94.00	61.00	92.00	56.00	40.00
TL_rec_on_OS_checkin	5993.00	5130.00	2090.00	2909.00	3468.00
week_since_last_OS_checkin	8.00	1.00	6.00	1.00	9.00
following_CP	Yes	No	Yes	Yes	No
working_flag	No	Yes	No	No	No
travelling_network_rating	1.00	4.00	2.00	3.00	4.00
Adult_flag	0.00	1.00	0.00	0.00	1.00
DA_mins_spend_on_CP	8.00	10.00	7.00	8.00	6.00

In [61]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Int64Index: 10455 entries, 0 to 11759
Data columns (total 15 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   Taken_product                10455 non-null  object
 1   YA_view_on_CP                10455 non-null  float64
 2   pref_device                  10455 non-null  object
 3   TL_done_on_OS_checkin        10455 non-null  float64
 4   YA_OS_checkin                10455 non-null  float64
 5   members_in_fam               10455 non-null  float64
 6   pref_loc_type                10455 non-null  object
 7   YA_comment_on_CP             10455 non-null  float64
 8   TL_rec_on_OS_checkin         10455 non-null  float64
 9   week_since_last_OS_checkin   10455 non-null  float64
 10  following_CP                 10455 non-null  object
 11  working_flag                 10455 non-null  object
 12  travelling_network_rating    10455 non-null  float64
 13  Adult_flag                   10455 non-null  float64
 14  DA_mins_spend_on_CP          10455 non-null  float64
dtypes: float64(10), object(5)
memory usage: 1.5+ MB
```

We have to build models separately for Laptop and Mobile users : Let's make 2 different dataframes for each :-

So, let us first split the dataset into two seperate dataframes.

In [62]:
```
# data_laptop is the dataset of the users who preferred laptop

data_laptop = data[data['pref_device'] == 'Laptop']


# data_mobile is the dataset of the users who preferred mobile

data_mobile = data[data['pref_device'] == 'Mobile']
```

In [63]:
```
print('Shape of the dataset:',data_laptop.shape)

data_laptop.head().T
```

```
Shape of the dataset: (1108, 15)
```

Out[63]:

	5881	5883	5885	5888	5906
Taken_product	No	No	No	No	Yes
YA_view_on_CP	379.00	255.00	250.00	295.00	250.00
pref_device	Laptop	Laptop	Laptop	Laptop	Laptop
TL_done_on_OS_checkin	9776.00	48725.00	35184.00	7572.00	39379.00
YA_OS_checkin	8.00	3.00	2.00	23.00	16.00
members_in_fam	1.00	4.00	2.00	3.00	1.00
pref_loc_type	Adventure and Tourism	Adventure and Tourism	Adventure and Tourism	Adventure and Tourism	Adventure and Tourism

	5881	5883	5885	5888	5906
YA_comment_on_CP	64.00	59.00	84.00	48.00	80.00
TL_rec_on_OS_checkin	5142.00	2913.00	3074.00	9532.00	3926.00
week_since_last_OS_checkin	1.00	1.00	0.00	0.00	1.00
following_CP	No	Yes	No	No	Yes
working_flag	Yes	No	No	Yes	Yes
travelling_network_rating	4.00	3.00	3.00	2.00	2.00
Adult_flag	1.00	0.00	0.00	0.00	0.00
DA_mins_spend_on_CP	10.00	8.00	8.00	10.00	10.00

In [224…

```
data_laptop.describe().T
```

Out[224…

	count	mean	std	min	25%	50%	75%	max
YA_view_on_CP	1108.00	282.13	64.24	138.00	237.00	270.50	325.00	463.00
TL_done_on_OS_checkin	1108.00	27769.66	13361.84	3818.00	17650.25	26942.50	39380.25	52512.00
YA_OS_checkin	1108.00	9.28	8.41	1.00	2.00	7.00	15.00	29.00
members_in_fam	1108.00	2.90	1.04	1.00	2.00	3.00	4.00	5.00
YA_comment_on_CP	1108.00	77.50	21.04	33.00	60.00	77.00	95.00	125.00
TL_rec_on_OS_checkin	1108.00	6130.68	4105.79	1098.00	2884.00	4759.50	7979.50	16572.00
week_since_last_OS_checkin	1108.00	3.00	2.49	0.00	1.00	2.00	4.00	11.00
travelling_network_rating	1108.00	2.78	1.08	1.00	2.00	3.00	4.00	4.00
Adult_flag	1108.00	0.56	0.50	0.00	0.00	1.00	1.00	1.00
DA_mins_spend_on_CP	1108.00	13.54	7.93	0.00	8.00	11.50	18.00	33.00

In [64]:

```
print('Shape of the dataset:',data_mobile.shape)

data_mobile.head().T
```

```
Shape of the dataset: (9347, 15)
```

Out[64]:

	0	1	2	3	4
Taken_product	Yes	No	Yes	No	No
YA_view_on_CP	307.00	367.00	277.00	247.00	202.00
pref_device	Mobile	Mobile	Mobile	Mobile	Mobile
TL_done_on_OS_checkin	38570.00	9765.00	48055.00	48720.00	20685.00
YA_OS_checkin	1.00	1.00	1.00	1.00	1.00
members_in_fam	2.00	1.00	2.00	4.00	1.00

	0	1	2	3	4
pref_loc_type	Financial	Financial	Other	Financial	Medical
YA_comment_on_CP	94.00	61.00	92.00	56.00	40.00
TL_rec_on_OS_checkin	5993.00	5130.00	2090.00	2909.00	3468.00
week_since_last_OS_checkin	8.00	1.00	6.00	1.00	9.00
following_CP	Yes	No	Yes	Yes	No
working_flag	No	Yes	No	No	No
travelling_network_rating	1.00	4.00	2.00	3.00	4.00
Adult_flag	0.00	1.00	0.00	0.00	1.00
DA_mins_spend_on_CP	8.00	10.00	7.00	8.00	6.00

In [222…
```
data_mobile.describe().T
```

Out[222…

	count	mean	std	min	25%	50%	75%	max
YA_view_on_CP	9347.00	281.21	68.60	92.50	232.00	271.00	325.00	464.00
TL_done_on_OS_checkin	9347.00	28186.09	14215.68	3570.00	16240.00	28294.00	40672.00	76843.25
YA_OS_checkin	9347.00	8.17	8.58	1.00	1.00	4.00	14.00	29.00
members_in_fam	9347.00	2.92	1.02	1.00	2.00	3.00	4.00	7.00
YA_comment_on_CP	9347.00	74.76	21.85	3.00	57.00	75.00	93.00	147.00
TL_rec_on_OS_checkin	9347.00	6421.30	4368.67	1009.00	2960.50	4968.00	8398.50	16572.00
week_since_last_OS_checkin	9347.00	3.23	2.63	0.00	1.00	3.00	5.00	11.00
travelling_network_rating	9347.00	2.71	1.08	1.00	2.00	3.00	4.00	4.00
Adult_flag	9347.00	0.57	0.49	0.00	0.00	1.00	1.00	1.00
DA_mins_spend_on_CP	9347.00	13.69	7.99	0.00	8.00	12.00	18.00	33.00

Data Encoding for Modeling Purpose : Now, we will do 'One Hot Encoding' for the object type variables :-

In [65]:
```
data_laptop_dummy = pd.get_dummies(data_laptop, drop_first=True)

data_laptop_dummy.head().T
```

Out[65]:

	5881	5883	5885	5888	5906
YA_view_on_CP	379.00	255.00	250.00	295.00	250.00
TL_done_on_OS_checkin	9776.00	48725.00	35184.00	7572.00	39379.00
YA_OS_checkin	8.00	3.00	2.00	23.00	16.00

	5881	5883	5885	5888	5906
members_in_fam	1.00	4.00	2.00	3.00	1.00
YA_comment_on_CP	64.00	59.00	84.00	48.00	80.00
TL_rec_on_OS_checkin	5142.00	2913.00	3074.00	9532.00	3926.00
week_since_last_OS_checkin	1.00	1.00	0.00	0.00	1.00
travelling_network_rating	4.00	3.00	3.00	2.00	2.00
Adult_flag	1.00	0.00	0.00	0.00	0.00
DA_mins_spend_on_CP	10.00	8.00	8.00	10.00	10.00
Taken_product_Yes	0.00	0.00	0.00	0.00	1.00
pref_loc_type_Big Cities	0.00	0.00	0.00	0.00	0.00
pref_loc_type_Other	0.00	0.00	0.00	0.00	0.00
following_CP_Yes	0.00	1.00	0.00	0.00	1.00
working_flag_Yes	1.00	0.00	0.00	1.00	1.00

In [66]:
```
data_mobile_dummy = pd.get_dummies(data_mobile, drop_first=True)

data_mobile_dummy.head().T
```

Out[66]:

	0	1	2	3	4
YA_view_on_CP	307.00	367.00	277.00	247.00	202.00
TL_done_on_OS_checkin	38570.00	9765.00	48055.00	48720.00	20685.00
YA_OS_checkin	1.00	1.00	1.00	1.00	1.00
members_in_fam	2.00	1.00	2.00	4.00	1.00
YA_comment_on_CP	94.00	61.00	92.00	56.00	40.00
TL_rec_on_OS_checkin	5993.00	5130.00	2090.00	2909.00	3468.00
week_since_last_OS_checkin	8.00	1.00	6.00	1.00	9.00
travelling_network_rating	1.00	4.00	2.00	3.00	4.00
Adult_flag	0.00	1.00	0.00	0.00	1.00
DA_mins_spend_on_CP	8.00	10.00	7.00	8.00	6.00
Taken_product_Yes	1.00	0.00	1.00	0.00	0.00
pref_loc_type_Big Cities	0.00	0.00	0.00	0.00	0.00
pref_loc_type_Entertainment	0.00	0.00	0.00	0.00	0.00
pref_loc_type_Financial	1.00	1.00	0.00	1.00	0.00
pref_loc_type_Medical	0.00	0.00	0.00	0.00	1.00
pref_loc_type_Other	0.00	0.00	1.00	0.00	0.00
following_CP_Yes	1.00	0.00	1.00	1.00	0.00

	0	1	2	3	4
working_flag_Yes	0.00	1.00	0.00	0.00	0.00

Calculations for Model Building : For Laptop Users

Train-Test Split :-

In [67]:
```
#Now, let's separate dependent and independent variables

X = data_laptop_dummy.drop('Taken_product_Yes', axis=1)

Y = data_laptop_dummy[['Taken_product_Yes']]
```

In [68]:
```
# Split the data into test and train sets in 70:30 ratio

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.30, random_stat
```

In [208…
```
print("{0:0.2f}% data is in training set".format((len(X_train)/len(data_laptop_dummy.in
print("{0:0.2f}% data is in test set".format((len(X_test)/len(data_laptop_dummy.index))
```

```
69.95% data is in training set
30.05% data is in test set
```

In [69]:
```
print('Shape of X_train:',X_train.shape)

X_train.head().T
```

```
Shape of X_train: (775, 14)
```

Out[69]:

	9580	8684	6956	11043	9186
YA_view_on_CP	409.00	214.00	408.00	307.00	321.00
TL_done_on_OS_checkin	4393.00	39588.00	36753.00	33122.00	5040.00
YA_OS_checkin	2.00	15.00	11.00	22.00	4.00
members_in_fam	3.00	3.00	4.00	3.00	3.00
YA_comment_on_CP	54.00	80.00	90.00	108.00	73.00
TL_rec_on_OS_checkin	7586.00	4785.00	16038.00	10930.00	9418.00
week_since_last_OS_checkin	2.00	0.00	3.00	1.00	1.00
travelling_network_rating	1.00	4.00	1.00	3.00	3.00
Adult_flag	1.00	1.00	1.00	0.00	0.00
DA_mins_spend_on_CP	33.00	8.00	26.00	23.00	10.00
pref_loc_type_Big Cities	1.00	0.00	0.00	0.00	1.00
pref_loc_type_Other	0.00	0.00	0.00	0.00	0.00
following_CP_Yes	0.00	0.00	0.00	1.00	0.00

	9580	8684	6956	11043	9186
working_flag_Yes	0.00	0.00	1.00	0.00	0.00

In [70]:
```
# Normalization of X_train

normalized_X_train = pd.DataFrame(StandardScaler().fit_transform(X_train),columns = X_t
```

In [71]:
```
print('Shape of y_train:',Y_train.shape)

Y_train.head().T
```

```
Shape of y_train: (775, 1)
```

Out[71]:

	9580	8684	6956	11043	9186
Taken_product_Yes	0	0	0	0	1

In [72]:
```
print('Shape of X_test:',X_test.shape)

X_test.head().T
```

```
Shape of X_test: (333, 14)
```

Out[72]:

	9905	6087	7577	9102	7807
YA_view_on_CP	238.00	280.00	226.00	227.00	143.00
TL_done_on_OS_checkin	19691.00	22230.00	49466.00	15525.00	45722.00
YA_OS_checkin	3.00	18.00	1.00	2.00	5.00
members_in_fam	3.00	1.00	1.00	2.00	3.00
YA_comment_on_CP	50.00	76.00	51.00	61.00	73.00
TL_rec_on_OS_checkin	4147.00	2164.00	7929.00	4620.00	1885.00
week_since_last_OS_checkin	1.00	1.00	1.00	1.00	1.00
travelling_network_rating	1.00	1.00	4.00	1.00	4.00
Adult_flag	0.00	0.00	1.00	0.00	0.00
DA_mins_spend_on_CP	12.00	8.00	11.00	17.00	0.00
pref_loc_type_Big Cities	0.00	0.00	0.00	0.00	1.00
pref_loc_type_Other	0.00	0.00	0.00	0.00	0.00
following_CP_Yes	0.00	0.00	0.00	0.00	1.00
working_flag_Yes	0.00	0.00	0.00	0.00	0.00

In [73]:
```
# Normalization of X_test

normalized_X_test = pd.DataFrame(StandardScaler().fit_transform(X_test),columns = X_tes
```

In [74]:
```
print('Shape of y_test:',Y_test.shape)

Y_test.head()
```

```
Shape of y_test: (333, 1)
```

Out[74]:

	Taken_product_Yes
9905	1
6087	0
7577	0
9102	0
7807	1

In [75]:
```
data_laptop_dummy['Taken_product_Yes'].value_counts()
```

Out[75]:
```
0    832
1    276
Name: Taken_product_Yes, dtype: int64
```

832 laptop users has not taken the product and only 276 users has taken the product. This means that there is class imbalance in the dataset, where 1 is minority class and 0 is in majority class.

Need of applying SMOTE :-

As per our analysis, we got to know that the data is more towards the negative side, like majority of the users have not taken our product.This clearly means that data is in imbalance state.

Standard ML techniques such as Decision Tree and Logistic Regression have a bias towards the majority class, and they tend to ignore the minority class. They tend only to predict the majority class, hence, having major misclassification of the minority class in comparison with the majority class. In more technical words, if we have imbalanced data distribution in our dataset then our model becomes more prone to the case when minority class has negligible or very lesser recall.

So we will be using a over sampling technique called SMOTE to handle imbalanced data.

Logistic Regression Model (with default values) : On imbalance data

In [76]:
```
LR = LogisticRegression()

# Now let's fit the model on training data

LR.fit(normalized_X_train, Y_train)
```

Out[76]:
```
LogisticRegression()
```

In [226…
```
# Prdecition on training data

Y_train_predict = LR.predict(normalized_X_train)
```

In [78]:
```
# Prdecition on testing data

Y_test_predict = LR.predict(normalized_X_test)
```

In [228…
```
# Getting the predicted classes and probs

Y_test_predict_prob = LR.predict_proba(normalized_X_test)
```

In [80]:
```
# Model Evaluation - Accuracy on training data

LR.score(normalized_X_train, Y_train)
```

Out[80]:
```
0.8490322580645161
```

In [81]:
```
# Classification Report - training data

print(classification_report(Y_train, Y_train_predict))
```

```
              precision    recall  f1-score   support

           0       0.86      0.95      0.91       588
           1       0.78      0.52      0.62       187

    accuracy                           0.85       775
   macro avg       0.82      0.74      0.76       775
weighted avg       0.84      0.85      0.84       775
```

In [82]:
```
# Classification Report - testing data

print(classification_report(Y_test, Y_test_predict))
```

```
              precision    recall  f1-score   support

           0       0.80      0.93      0.86       244
           1       0.67      0.38      0.49        89

    accuracy                           0.78       333
   macro avg       0.74      0.66      0.67       333
weighted avg       0.77      0.78      0.76       333
```

We can clearly see that the accuracy comes out to be 78% in test data and 85% in train data, but, the recall of the minority class in very less(38%). It means that the model is more biased towards majority class. So, it proves that this is not the best model, and it might be because of class imbalance.

Applying SMOTE :-

Now, we will apply the imbalanced data handling technique, called SMOTE and see their accuracy and recall results.

In [83]:
```

```

```
SM = SMOTE(random_state = 2)

X_train_SM, Y_train_SM = SM.fit_sample(normalized_X_train, Y_train)
```

In [84]:
```
print('After OverSampling, the shape of X_train:',X_train_SM.shape)
print('After OverSampling, the shape of Y_train:',Y_train_SM.shape)
```

```
After OverSampling, the shape of X_train: (1176, 14)
After OverSampling, the shape of Y_train: (1176, 1)
```

Logistic Regression Model (with default values) : On balanced data - After applying SMOTE - Oversampling

Now we will again make the model again and check the recall values for majority and minority classes

In [85]:
```
LR_SM = LogisticRegression()

# Now let's fit the model on train data

LR_SM.fit(X_train_SM, Y_train_SM)
```

Out[85]:
```
LogisticRegression()
```

In [86]:
```
# Prdecition on training data

Y_train_predict_SM = LR_SM.predict(X_train_SM)
```

In [87]:
```
# Prdecition on testing data

Y_test_predict_SM = LR_SM.predict(normalized_X_test)
```

In [88]:
```
# Model Evaluation - Accuracy on training data

LR_SM.score(X_train_SM, Y_train_SM)
```

Out[88]:
```
0.7874149659863946
```

In [89]:
```
# Classification Report - training data

print(classification_report(Y_train_SM, Y_train_predict_SM))
```

```
              precision    recall  f1-score   support

           0       0.80      0.77      0.78       588
           1       0.78      0.80      0.79       588

    accuracy                           0.79      1176
   macro avg       0.79      0.79      0.79      1176
weighted avg       0.79      0.79      0.79      1176
```

In [90]:
```
# Classification Report - testing data

print(classification_report(Y_test, Y_test_predict_SM))
```

```
              precision    recall  f1-score   support

           0       0.87      0.75      0.80       244
           1       0.50      0.70      0.58        89

    accuracy                           0.73       333
   macro avg       0.69      0.72      0.69       333
weighted avg       0.77      0.73      0.74       333
```

Now the difference is clear, that after over-sampling, the accuracy has been decreased from 85% to 73%.

But as we know that the accuracy is not the good predictor here in our case because we are dealing with less classes and it is also a classification algorithm.

So here we will focus more on improving our recall. Now, the recall for the minority class has been increased a lot, and it is 70%. That is very good. But the recall for the majority class has been decreased, this is due the fact that now that there is a balance in both the classes.

Recall value for the minority class(1) is 0.70 which is a good prediction, since our model has predicted 70% of the 1's as 1's and only 30% of the prediction were wrong.

Logistic Regression Model (Customized Values) : On balanced data - After applying SMOTE - Oversampling

As we have built the previous model with it's default values. There might be chances of better model than this, with some customized parameters for the logistic regression model.

We can improve the model by tuning the hyper-parameters of the model. So here we will use some custom parameters and then check our model performance.

In [91]:
```
# Fit the Logistic Regression model

LR_SM_custom = LogisticRegression(solver='lbfgs', penalty='none', max_iter=100000, n_jo

LR_SM_custom.fit(X_train_SM, Y_train_SM)
```

Out[91]:
```
[Parallel(n_jobs=2)]: Using backend LokyBackend with 2 concurrent workers.
[Parallel(n_jobs=2)]: Done   1 out of   1 | elapsed:    1.8s finished
LogisticRegression(max_iter=100000, n_jobs=2, penalty='none', verbose=True)
```

In [92]:
```
# Prdecition on training data

Y_train_predict_SM_custom = LR_SM_custom.predict(X_train_SM)
```

In [93]:

```
# Prdecition on testing data

Y_test_predict_SM_custom = LR_SM_custom.predict(normalized_X_test)
```

In [94]:
```
# Model Evaluation - Accuracy on training data

LR_SM_custom.score(X_train_SM, Y_train_SM)
```

Out[94]:
```
0.7882653061224489
```

In [95]:
```
# Classification Report - training data

print(classification_report(Y_train_SM, Y_train_predict_SM_custom))
```

```
              precision    recall  f1-score   support

           0       0.80      0.77      0.79       588
           1       0.78      0.80      0.79       588

    accuracy                           0.79      1176
   macro avg       0.79      0.79      0.79      1176
weighted avg       0.79      0.79      0.79      1176
```

In [96]:
```
# Classification Report - testing data

print(classification_report(Y_test, Y_test_predict_SM_custom))
```

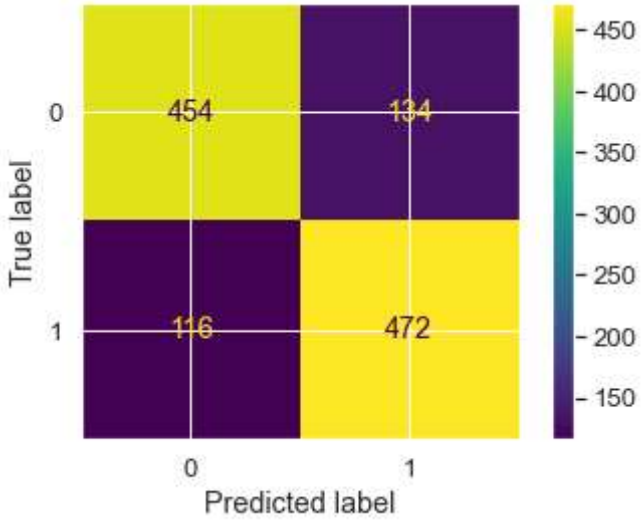
```
              precision    recall  f1-score   support

           0       0.87      0.75      0.80       244
           1       0.50      0.70      0.58        89

    accuracy                           0.73       333
   macro avg       0.69      0.72      0.69       333
weighted avg       0.77      0.73      0.74       333
```

There is not much difference in the matrix using customized parameters.

Confusion Matrix, AUC, ROC for the Model :-

In [97]:
```
# Confusion matrix on the training data

plot_confusion_matrix(LR_SM, X_train_SM, Y_train_SM)
```

Out[97]:
```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x2364ee34610>
```

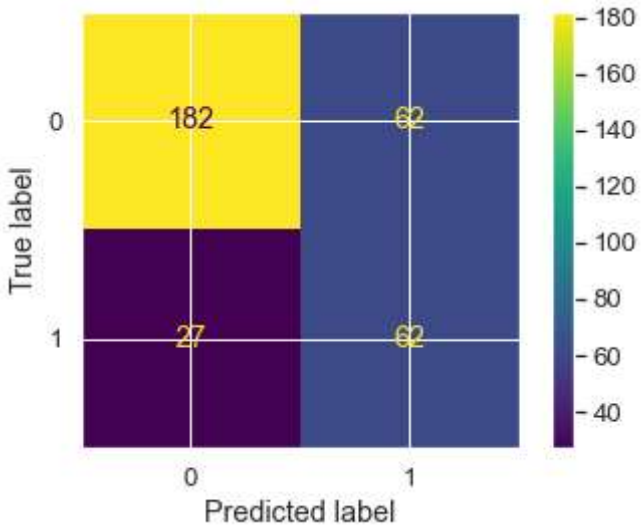


So, True Positives are coming here is 472, that means 472 times our model has predicted correctly, that means it has predicted 1 as 1, and only 116 times it has predicted incorrectly.

Similarly, True Negatives are 454 that means 454 times it has predicted 0 as 0 and only 116 times, the prediction was incorrect.

In [98]:
```
# Confusion Matrix - Test data

plot_confusion_matrix(LR_SM, normalized_X_test, Y_test)
```

Out[98]:
```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x2364ee73130>
```



So, in test data, True Positives are coming here is 62, that means 62 times our model has predicted correctly, that means it has predicted 1 as 1, and only 27 times it has predicted incorrectly.

Similarly, True Negatives are 182 that means 182 times it has predicted 0 as 0 and only 62 times, the prediction was incorrect.

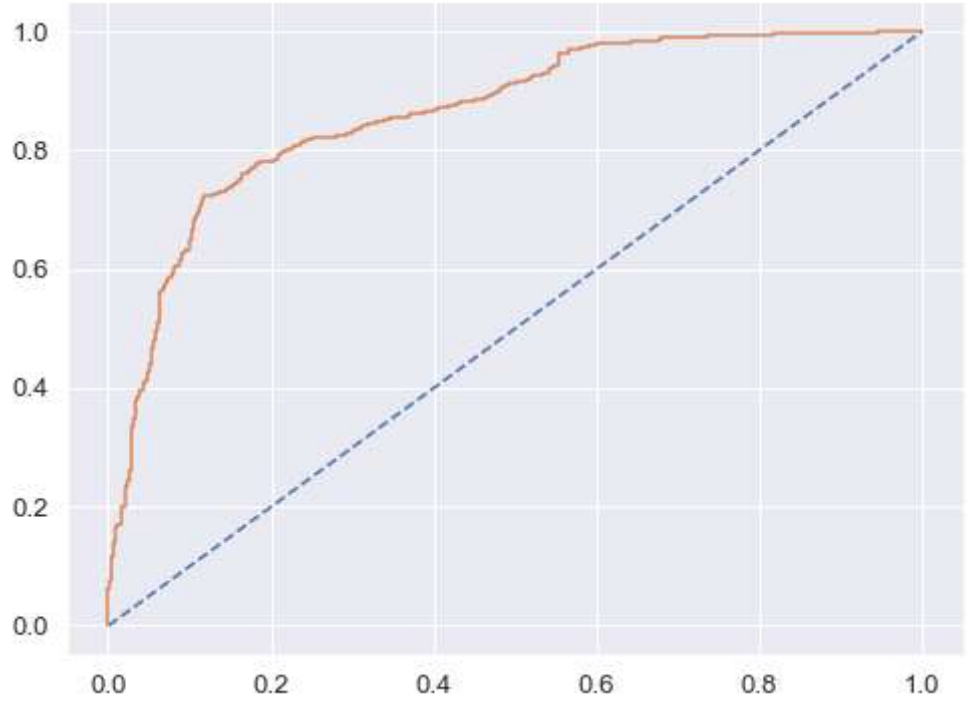
In [99]:
```
# Predicting Probabilities

probs = LR_SM.predict_proba(X_train_SM)
```

In [100…
```
# keep probabilities for the positive outcome only

probs = probs[:, 1]
```

In [101…
```
# Calculate Area Under Curve

auc = roc_auc_score(Y_train_SM, probs)

print('AUC: %.3f' % auc)
```

```
AUC: 0.862
```

In [102…
```
# Calculate ROC curve

train_fpr, train_tpr, train_thresholds = roc_curve(Y_train_SM, probs)
plt.rcParams["figure.figsize"] = (8,6)
plt.plot([0, 1], [0, 1], linestyle='--')


# plot the roc curve for the model
plt.plot(train_fpr, train_tpr);
```



In [103…
```
#Accuracy score - for Test data

acc_test = LR_SM.score(normalized_X_test, Y_test)

acc_test
```

Out[103…
```
0.7327327327327328
```

So in the above model, the area under the curve is 86% that means the overall model performance

is good.

Also, the confusion matrix and classification report the LR_SM Logistic Regression Model is giving the good results.

Using GridSearchCV for Logistic Regression :-

In [104…
```
grid = {'penalty':['l2','none'], 'solver':['sag','lbfgs'], 'tol':[0.0001,0.00001]}

LR_grid = LogisticRegression(max_iter=10000, n_jobs=2)
```

In [105…
```
grid_search = GridSearchCV(estimator=LR_grid, param_grid = grid, cv=3, n_jobs=-1, scori

grid_search.fit(X_train_SM, Y_train_SM)
```

Out[105…
```
GridSearchCV(cv=3, estimator=LogisticRegression(max_iter=10000, n_jobs=2),
             n_jobs=-1,
             param_grid={'penalty': ['l2', 'none'], 'solver': ['sag', 'lbfgs'],
                         'tol': [0.0001, 1e-05]},
             scoring='f1')
```

In [106…
```
print('Best Parameters from GridSearch:',grid_search.best_params_)

print('\nBest estimator from GridSearch:',grid_search.best_estimator_)
```

```
Best Parameters from GridSearch: {'penalty': 'l2', 'solver': 'sag', 'tol': 0.0001}

Best estimator from GridSearch: LogisticRegression(max_iter=10000, n_jobs=2, solver='sa
g')
```

In [107…
```
best_LR_model = grid_search.best_estimator_
```

In [108…
```
# Prediction - on Train data

Y_train_grid_predict = best_LR_model.predict(X_train_SM)

Y_test_grid_predict = best_LR_model.predict(normalized_X_test)
```

In [109…
```
# Predicting Probabilities - on Test data

Y_test_grid_predict_prob = best_LR_model.predict_proba(normalized_X_test)

pd.DataFrame(Y_test_grid_predict_prob).head()
```

Out[109…

	0	1
0	0.60	0.40
1	0.23	0.77
2	0.99	0.01
3	0.67	0.33

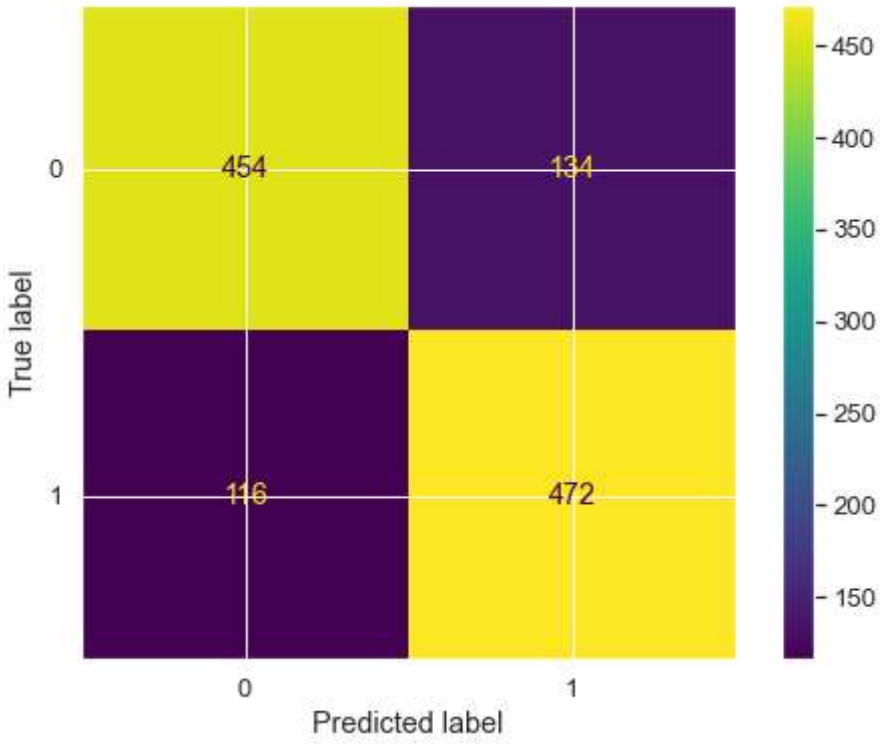
	0	1
4	0.21	0.79

In [110…

```
# Confusion Matrix - Train data

plot_confusion_matrix(best_LR_model, X_train_SM, Y_train_SM)
```

Out[110…

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x2364c8a79a0>
```



In [111…

```
# Classification Report - Train data

print(classification_report(Y_train_SM, Y_train_grid_predict))
```

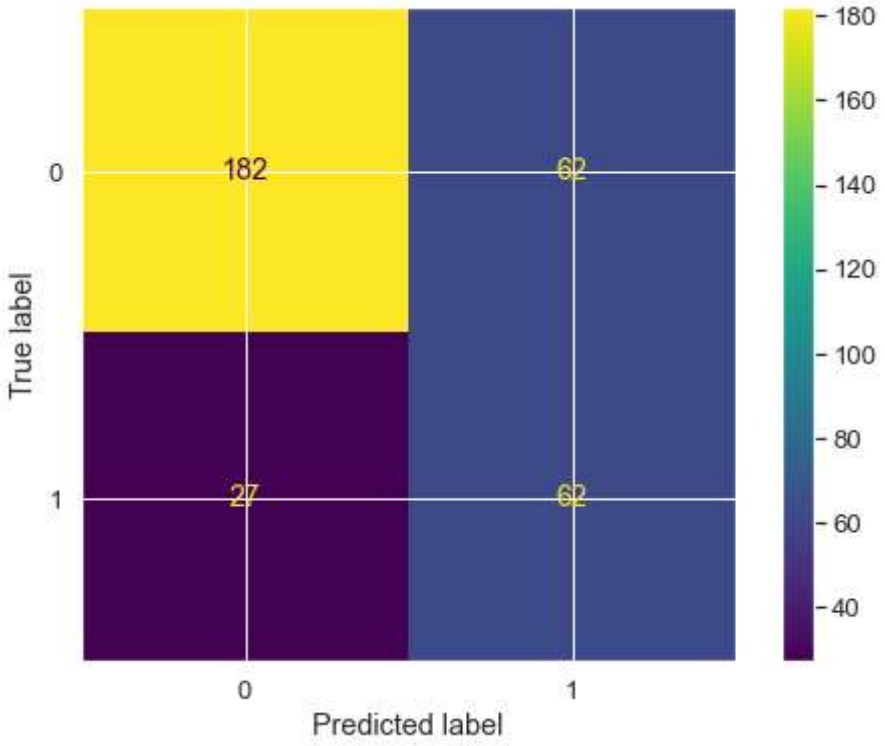
```
              precision    recall  f1-score   support

           0       0.80      0.77      0.78       588
           1       0.78      0.80      0.79       588

    accuracy                           0.79      1176
   macro avg       0.79      0.79      0.79      1176
weighted avg       0.79      0.79      0.79      1176
```

In [112…

```
# Confusion Matrix - Test data

plot_confusion_matrix(best_LR_model, normalized_X_test, Y_test)
```

Out[112…

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x2364f025ca0>
```



In [113…
```
# Classification Report - Test data

print(classification_report(Y_test, Y_test_grid_predict))
```

```
              precision    recall  f1-score   support

           0       0.87      0.75      0.80       244
           1       0.50      0.70      0.58        89

    accuracy                           0.73       333
   macro avg       0.69      0.72      0.69       333
weighted avg       0.77      0.73      0.74       333
```

There is no change in the recall values and other matrices as well after using GridSearchCV.

This might not be the best model. We will check other models as well.

KNN Model (k-nearest neighbors)

We will be building KNN model on re-sampled train sets.

In [114…
```
# Importing KNeighboursClassifier

KNN_model = KNeighborsClassifier()

KNN_model.fit(X_train_SM, Y_train_SM)
```

Out[114… KNeighborsClassifier()

In [230…
```
# Predicting the model on Train set and check the model score
```

```
predicted_labels = KNN_model.predict(X_train_SM)

KNN_model_score = KNN_model.score(X_train_SM, Y_train_SM)
```

In [231…

```
predicted_labels_test = KNN_model.predict(normalized_X_test)

KNN_model_score_test = KNN_model.score(normalized_X_test, Y_test)
```

In [116…

```
KNN_model_score
```

Out[116…

```
0.9914965986394558
```

In [232…

```
KNN_model_score_test
```

Out[232…

```
0.9159159159159159
```

Accuracy is quite good. Let see the other metrics.

Classification Report and Confusion Matrix :-

In [117…

```
# For train data

print('Confusion Matrix\n')

print(metrics.confusion_matrix(Y_train_SM, predicted_labels))

print('Classification Report\n')

print(metrics.classification_report(Y_train_SM, predicted_labels))
```

```
Confusion Matrix

[[578  10]
 [  0 588]]
Classification Report

              precision    recall  f1-score   support

           0       1.00      0.98      0.99       588
           1       0.98      1.00      0.99       588

    accuracy                           0.99      1176
   macro avg       0.99      0.99      0.99      1176
weighted avg       0.99      0.99      0.99      1176
```

In [233…

```
# For test data

print('Confusion Matrix\n')

print(metrics.confusion_matrix(Y_test, predicted_labels_test))

print('Classification Report\n')
```

```
print(metrics.classification_report(Y_test, predicted_labels_test))
```

```
Confusion Matrix

[[216  28]
 [  0  89]]
Classification Report

              precision    recall  f1-score   support

           0       1.00      0.89      0.94       244
           1       0.76      1.00      0.86        89

    accuracy                           0.92       333
   macro avg       0.88      0.94      0.90       333
weighted avg       0.94      0.92      0.92       333
```

Recall Value is much better than the logistic Regression model.

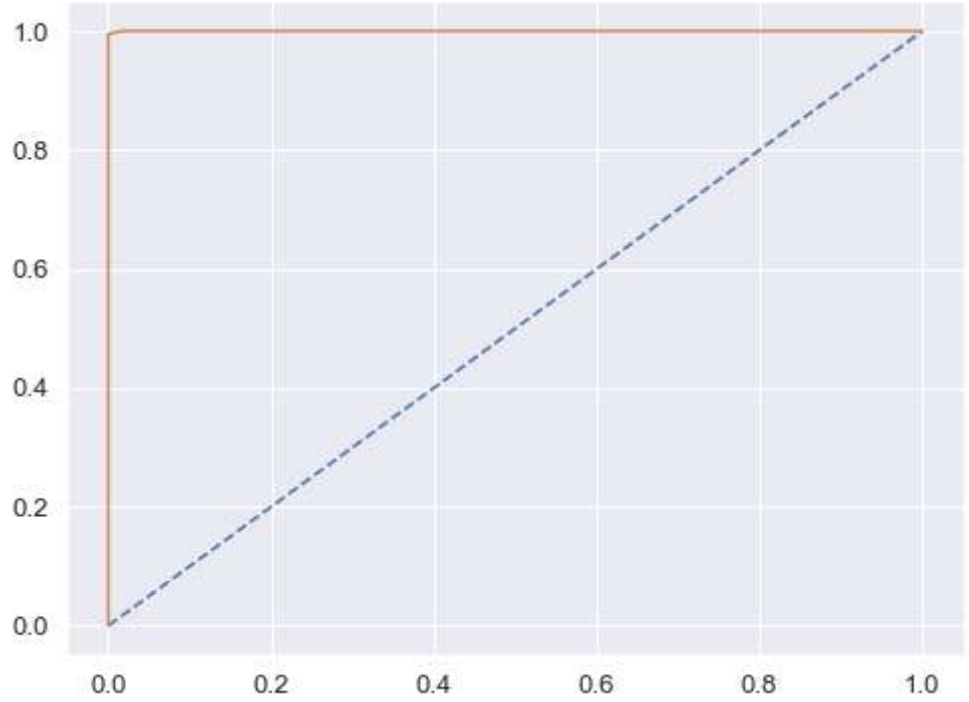
Recall for the minority class(1) is 80 which is good, and also the accuracy is 79 that is also a good score. Now let's check the AUC ROC Curve and other metrics as well.

AUC ROC Curve :-

In [118…

```
# AUC ROC Curve on Train data

probs_train = KNN_model.predict_proba(X_train_SM)

probs_train = probs_train[:, 1]

auc=roc_auc_score(Y_train_SM,probs_train)


print("Area Under Curve: %.3f " % auc)
train_fpr ,train_tpr, train_thresholds= roc_curve(Y_train_SM, probs_train)
plt.plot([0,1],[0,1],linestyle='--')
plt.plot(train_fpr,train_tpr);
```

```
Area Under Curve: 1.000
```



Since, the AUC = 100%, which is too good and it means that the KNN based classifier's performance is better for the model at distinguishing between the positive and negative classes.

Naive Bayes Model

In [119…
```
# Fit the model into training data

NB_model = GaussianNB()

NB_model.fit(X_train_SM, Y_train_SM)
```

Out[119…
```
GaussianNB()
```

Let see the performance of our model with training data :-

In [120…
```
NB_train_predict = NB_model.predict(X_train_SM)


print("Model Accuracy: {0:.2f}".format(metrics.accuracy_score(Y_train_SM, NB_train_pred
```

```
Model Accuracy: 0.73
```

Let see the performance of our model with testing data :-

In [121…
```
NB_test_predict = NB_model.predict(normalized_X_test)


print("Model Accuracy: {0:.2f}".format(metrics.accuracy_score(Y_test, NB_test_predict))
```

```
Model Accuracy: 0.70
```

Confusion Matrix & Classification Report :-

In [122…

```
print("Confusion Matrix:\n")

CM = metrics.confusion_matrix(Y_test, NB_test_predict, labels=[1, 0])

df_CM = pd.DataFrame(CM, index = [i for i in ["1","0"]],
                  columns = [i for i in ["Predict 1","Predict 0"]])


# Plotting the matrix
plt.figure(figsize = (8,6))
sns.heatmap(df_CM, annot=True)
```

```
Confusion Matrix:
```

Out[122…

```
<AxesSubplot:>
```

In [123…

```
print("Classification Report")
print(metrics.classification_report(Y_test, NB_test_predict, labels=[1, 0]))
```

```
Classification Report
              precision    recall  f1-score   support

           1       0.46      0.74      0.57        89
           0       0.88      0.69      0.77       244

    accuracy                           0.70       333
   macro avg       0.67      0.72      0.67       333
weighted avg       0.77      0.70      0.72       333
```

2. Model Tuning

1. Ensemble modeling, wherever applicable.

2. Any other model tuning measures(if applicable).
3. Interpretation of the most optimum model and its implication on the business.

Random Forest

In [124...
```
rfcl = RandomForestClassifier(n_estimators=301, oob_score = True, max_depth = 10, max_f

rfcl = rfcl.fit(X_train_SM, Y_train_SM)
```

n_estimators = number of trees, that we want to build within the RandomForestClassifier.

Out of bag (OOB) score is a way of validating the Random forest model. OOB score is computed as the number of correctly predicted rows from the out of bag sample.

max_features will be used to select randomly features from the available number of features.

In [125...
```
rfcl
```

Out[125...
```
RandomForestClassifier(max_depth=10, max_features=5, min_samples_leaf=50,
                       min_samples_split=110, n_estimators=301, oob_score=True)
```

In [126...
```
rfcl.oob_score_
```

Out[126...
```
0.826530612244898
```

In [127...
```
#Checking the error rate in %:

print('Error_rate(in %) :', 1- (rfcl.oob_score_)) #OOB Error is the number of wrongly c
```

```
Error_rate(in %) : 0.173469387755102
```

For the above specified grid parameters, we got the rfcl model with oob score around 83% and error rate 17%, which shows the model is good to go for further evaluation.

Getting the Feature Importances :-

In [128...
```
print (pd.DataFrame(rfcl.feature_importances_, columns = ["Imp"],
                    index = X_train_SM.columns).sort_values('Imp',ascending=False))
```

```
                            Imp
YA_view_on_CP              0.21
TL_rec_on_OS_checkin       0.18
following_CP_Yes           0.16
Adult_flag                 0.10
DA_mins_spend_on_CP        0.09
TL_done_on_OS_checkin      0.07
week_since_last_OS_checkin 0.05
travelling_network_rating  0.04
YA_OS_checkin              0.04
YA_comment_on_CP           0.04
members_in_fam             0.02
working_flag_Yes           0.00
```

```
pref_loc_type_Big Cities    0.00
pref_loc_type_Other         0.00
```

Let's Now use Ensemble techniques : Bagging, Ada Boosting, GradientBoost :-

In [129…

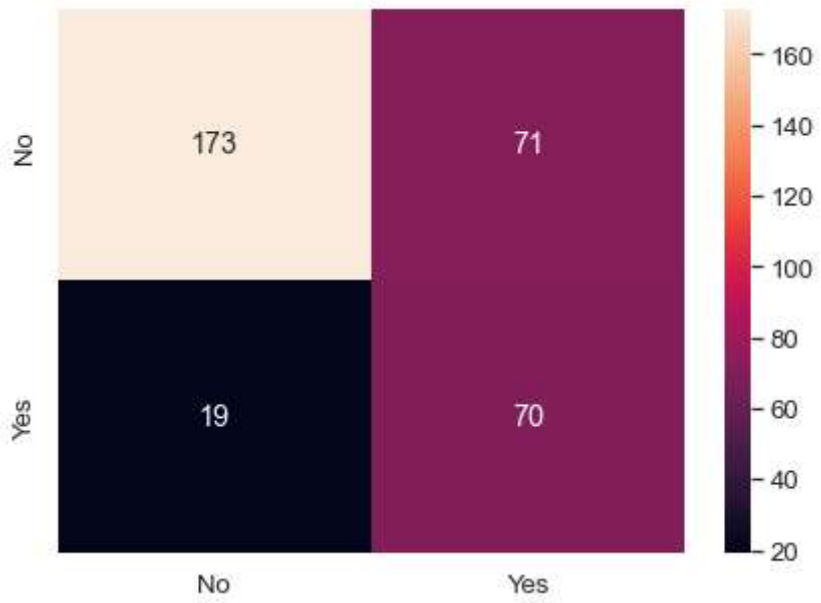
```
#Let's first look by use Bagging

#we will be using random forest as base_estimator

bgcl = BaggingClassifier(base_estimator=rfcl, n_estimators=50,random_state=1)


# Fit the model into training data
bgcl = bgcl.fit(X_train_SM, Y_train_SM)
```

In [130…

```
# Let's predict on test set

Y_test_predict = bgcl.predict(normalized_X_test)

print(bgcl.score(normalized_X_test , Y_test))

cm=metrics.confusion_matrix(Y_test, Y_test_predict,labels=[0, 1])

df_cm = pd.DataFrame(cm, index = [i for i in ["No","Yes"]],
                  columns = [i for i in ["No","Yes"]])

plt.figure(figsize = (7,5))
sns.heatmap(df_cm, annot=True ,fmt='g')
```

```
0.7297297297297297
<AxesSubplot:>
```

Out[130…



Ada Boosting

In [131…

```
# Defining the ada boosting classifier
```

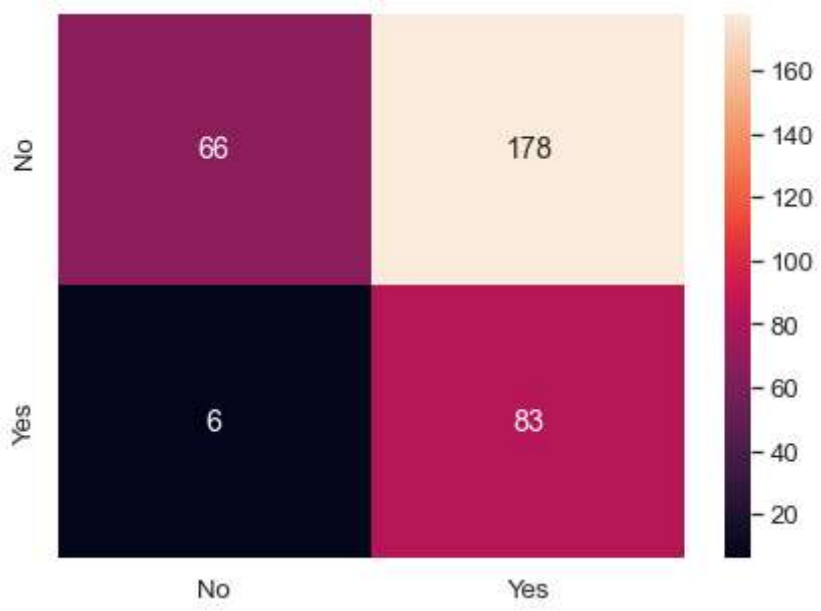
```
abcl = AdaBoostClassifier(n_estimators=10, random_state=1)

#fit the model to the train data
abcl = abcl.fit(X_train_SM, Y_train_SM)
```

In [132…

```
Y_predict_1 = abcl.predict(normalized_X_test)

print(abcl.score(normalized_X_test , Y_test))

cm = metrics.confusion_matrix(Y_test, Y_predict_1,labels=[0, 1])

df_cm_1 = pd.DataFrame(cm, index = [i for i in ["No","Yes"]],
                  columns = [i for i in ["No","Yes"]])
plt.figure(figsize = (7,5))
sns.heatmap(df_cm_1, annot=True ,fmt='g')
```

```
0.44744744744744747
<AxesSubplot:>
```

Out[132…



Gradient Boost

In [133…

```
gbcl = GradientBoostingClassifier(n_estimators = 50,random_state=1)
gbcl = gbcl.fit(X_train_SM, Y_train_SM)
```

In [134…

```
Y_predict_2 = gbcl.predict(normalized_X_test)
print(gbcl.score(normalized_X_test, Y_test))
```

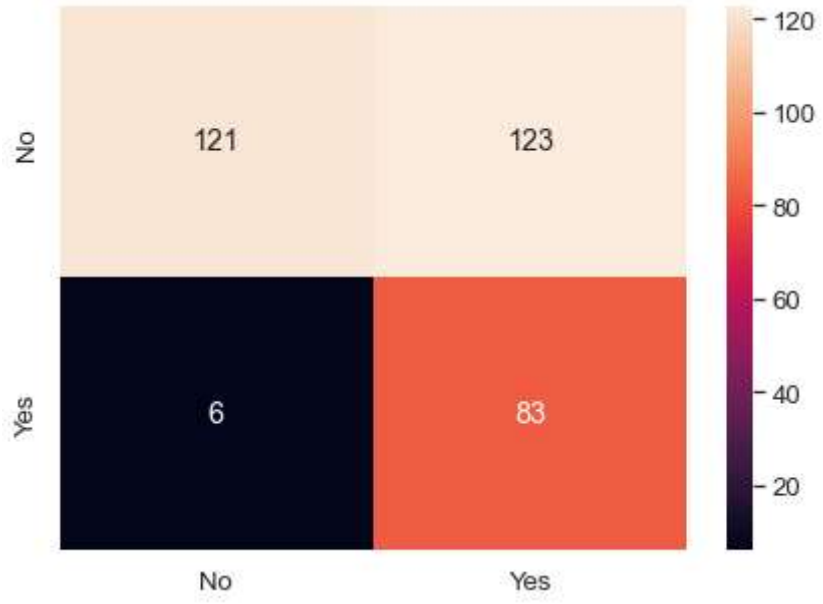
```
0.6126126126126126
```

In [135…

```
cm = metrics.confusion_matrix(Y_test, Y_predict_2,labels=[0, 1])

df_cm_2 = pd.DataFrame(cm, index = [i for i in ["No","Yes"]],
                  columns = [i for i in ["No","Yes"]])
```

```
plt.figure(figsize = (7,5))
sns.heatmap(df_cm_2, annot=True ,fmt='g')
```

Out[135… <AxesSubplot:>



Calculation for Model Building : For Mobile users

Train-Test Split :-

In [136…

```
#Now, let's separate dependent and independent variables

x = data_mobile_dummy.drop('Taken_product_Yes', axis=1)

y = data_mobile_dummy[['Taken_product_Yes']]
```

In [137…

```
# Split the data into test and train sets in 70:30 ratio

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.30, random_stat
```

In [138…

```
print('Shape of x_train:',x_train.shape)

x_train.head().T
```

```
Shape of x_train: (6542, 17)
```

Out[138…

	4471	10698	5645	8744	5377
YA_view_on_CP	287.00	403.00	346.00	245.00	301.00
TL_done_on_OS_checkin	22885.00	52192.00	44738.00	48062.00	11554.00
YA_OS_checkin	29.00	4.00	1.00	5.00	1.00
members_in_fam	5.00	2.00	3.00	4.00	4.00
YA_comment_on_CP	58.00	47.00	79.00	61.00	95.00

	4471	10698	5645	8744	5377
TL_rec_on_OS_checkin	2461.00	16572.00	10411.00	9687.00	2447.00
week_since_last_OS_checkin	1.00	4.00	6.00	8.00	3.00
travelling_network_rating	4.00	4.00	2.00	2.00	3.00
Adult_flag	0.00	0.00	1.00	0.00	1.00
DA_mins_spend_on_CP	15.00	33.00	19.00	8.00	7.00
pref_loc_type_Big Cities	0.00	0.00	0.00	0.00	0.00
pref_loc_type_Entertainment	0.00	0.00	0.00	0.00	0.00
pref_loc_type_Financial	1.00	0.00	1.00	0.00	1.00
pref_loc_type_Medical	0.00	0.00	0.00	0.00	0.00
pref_loc_type_Other	0.00	0.00	0.00	0.00	0.00
following_CP_Yes	0.00	0.00	1.00	0.00	0.00
working_flag_Yes	0.00	0.00	0.00	1.00	1.00

In [139…
```
# Normalization of x_train

normalized_x_train = pd.DataFrame(StandardScaler().fit_transform(x_train),columns = x_t
```

In [140…
```
print('Shape of y_train:',y_train.shape)

y_train.head().T
```

```
Shape of y_train: (6542, 1)
```

Out[140…

	4471	10698	5645	8744	5377
Taken_product_Yes	0	0	0	0	0

In [141…
```
print('Shape of x_test:',x_test.shape)

x_test.head().T
```

```
Shape of x_test: (2805, 17)
```

Out[141…

	8355	4104	6362	3270	11676
YA_view_on_CP	222.00	301.00	241.00	324.00	205.00
TL_done_on_OS_checkin	11627.00	15717.00	47778.00	19609.00	5545.00
YA_OS_checkin	17.00	16.00	13.00	10.00	1.00
members_in_fam	3.00	3.00	4.00	4.00	3.00
YA_comment_on_CP	55.00	99.00	51.00	96.00	111.00
TL_rec_on_OS_checkin	7995.00	7998.00	4237.00	5301.00	2924.00
week_since_last_OS_checkin	1.00	8.00	2.00	2.00	1.00

	8355	4104	6362	3270	11676
travelling_network_rating	1.00	4.00	3.00	2.00	4.00
Adult_flag	0.00	0.00	0.00	1.00	0.00
DA_mins_spend_on_CP	10.00	21.00	9.00	11.00	14.00
pref_loc_type_Big Cities	0.00	0.00	0.00	0.00	0.00
pref_loc_type_Entertainment	0.00	0.00	0.00	0.00	0.00
pref_loc_type_Financial	0.00	1.00	0.00	1.00	0.00
pref_loc_type_Medical	0.00	0.00	0.00	0.00	0.00
pref_loc_type_Other	1.00	0.00	0.00	0.00	0.00
following_CP_Yes	0.00	1.00	0.00	0.00	0.00
working_flag_Yes	0.00	0.00	0.00	0.00	0.00

In [142…
```
# Normalization of x_test

normalized_x_test = pd.DataFrame(StandardScaler().fit_transform(x_test),columns = x_tes
```

In [143…
```
print('Shape of y_test:',y_test.shape)

y_test.head()
```

```
Shape of y_test: (2805, 1)
```

Out[143…

	Taken_product_Yes
8355	0
4104	0
6362	1
3270	0
11676	0

In [144…
```
data_mobile_dummy['Taken_product_Yes'].value_counts()
```

Out[144…
```
0    7930
1    1417
Name: Taken_product_Yes, dtype: int64
```

Need of applying SMOTE :-

As per our analysis, we got to know that the data is more towards the negative side, like majority of the users have not taken our product.This clearly means that data is in imbalance state.

Standard ML techniques such as Decision Tree and Logistic Regression have a bias towards the majority class, and they tend to ignore the minority class. They tend only to predict the majority

class, hence, having major misclassification of the minority class in comparison with the majority class. In more technical words, if we have imbalanced data distribution in our dataset then our model becomes more prone to the case when minority class has negligible or very lesser recall.

So we will be using a over sampling technique called SMOTE to handle imbalanced data.

Logistic Regression Model (with default values) : On imbalance data

In [145…
```
# Let us first now train the data without handling imbalance problem

LR_mob = LogisticRegression()
```

In [146…
```
# Now let's fit the model on training data

LR_mob.fit(normalized_x_train, y_train)
```

Out[146… LogisticRegression()

In [147…
```
# Prdecition on training data

y_train_predict = LR_mob.predict(normalized_x_train)
```

In [148…
```
# Prdecition on testing data

y_test_predict = LR_mob.predict(normalized_x_test)
```

In [149…
```
# Getting the predicted classes and probs

y_test_predict_prob = LR_mob.predict_proba(normalized_x_test)
```

In [150…
```
# Model Evaluation - Accuracy on training data

LR_mob.score(normalized_x_train, y_train)
```

Out[150… 0.8720574747783553

In [151…
```
# Classification Report - training data

print(classification_report(y_train, y_train_predict))
```

```
              precision    recall  f1-score   support

           0       0.88      0.99      0.93      5550
           1       0.75      0.23      0.36       992

    accuracy                           0.87      6542
   macro avg       0.82      0.61      0.64      6542
weighted avg       0.86      0.87      0.84      6542
```

In [152…
```
# Classification Report - testing data

print(classification_report(y_test, y_test_predict))
```

```
              precision    recall  f1-score   support

           0       0.87      0.98      0.92      2380
           1       0.66      0.19      0.30       425

    accuracy                           0.86      2805
   macro avg       0.77      0.59      0.61      2805
weighted avg       0.84      0.86      0.83      2805
```

Applying SMOTE

Now, we will apply the imbalanced data handling technique, called SMOTE and test it's accuracy and recall results.

In [153…
```
SM = SMOTE(random_state = 2)

x_train_SM, y_train_SM = SM.fit_sample(normalized_x_train, y_train)
```

In [154…
```
print('After OverSampling, the shape of x_train:',x_train_SM.shape)

print('After OverSampling, the shape of y_train:',y_train_SM.shape)
```

```
After OverSampling, the shape of x_train: (11100, 17)
After OverSampling, the shape of y_train: (11100, 1)
```

Logistic Regression Model (with default values) : On balanced data - After applying SMOTE - Oversampling

Now we will again make the model again and check the recall values for majority and minority classes.

In [155…
```
LR_mob_SM = LogisticRegression()
```

In [156…
```
# Now let's fit the model on train data

LR_mob_SM.fit(x_train_SM, y_train_SM)
```

Out[156… `LogisticRegression()`

In [157…
```
# Prdecition - on Train data

y_train_predict_SM = LR_mob_SM.predict(x_train_SM)
```

In [158…
```
# Prdecition - on Test data
```

```
y_test_predict_SM = LR_mob_SM.predict(normalized_x_test)
```

In [159…

```
# Model Evaluation - Accuracy on Train data

LR_mob_SM.score(x_train_SM, y_train_SM)
```

Out[159…

```
0.7296396396396396
```

In [160…

```
# Classification Report - Train data

print(classification_report(y_train_SM, y_train_predict_SM))
```

```
              precision    recall  f1-score   support

           0       0.73      0.72      0.73      5550
           1       0.73      0.74      0.73      5550

    accuracy                           0.73     11100
   macro avg       0.73      0.73      0.73     11100
weighted avg       0.73      0.73      0.73     11100
```

In [161…

```
# Classification Report - Test data

print(classification_report(y_test, y_test_predict_SM))
```

```
              precision    recall  f1-score   support

           0       0.94      0.72      0.82      2380
           1       0.32      0.74      0.45       425

    accuracy                           0.72      2805
   macro avg       0.63      0.73      0.63      2805
weighted avg       0.85      0.72      0.76      2805
```

Now the difference is clear, that after over-sampling, the accuracy has been reduced, But as we know that the accuracy is not the good predictor here in our case because we are dealing with less classes and it is also a classification algorithm.

So here we will focus more on improving our recall. Now, the recall for the minority class has been increased a lot, and it is very good. But the recall for the majority class has been decreased, this is due the fact that now that there is a balance in both the classes.

Logistic Regression Model (Customized Values) : On balanced data - After applying SMOTE - Oversampling

As we have build the previous model with it's default values. There might be chances of more better model than this, with some customized parameters for the logistic regression model.

We can improve the model by tuning the hyper-parameters of the model. So here we will use 'GridSearchCV' method to get the best parameters and then we would want to test out in the model

again.

In [162…
```
# Fit the Logistic Regression model

LR_mob_SM_custom = LogisticRegression(solver='lbfgs', penalty='none', max_iter=100000,

LR_mob_SM_custom.fit(x_train_SM, y_train_SM)
```

```
[Parallel(n_jobs=2)]: Using backend LokyBackend with 2 concurrent workers.
[Parallel(n_jobs=2)]: Done   1 out of   1 | elapsed:    1.9s finished
```
Out[162… LogisticRegression(max_iter=100000, n_jobs=2, penalty='none', verbose=True)

In [163…
```
# Prdecition - on Train data

y_train_predict_SM_custom = LR_mob_SM_custom.predict(x_train_SM)
```

In [164…
```
# Prdecition - on Test data

y_test_predict_SM_custom = LR_mob_SM_custom.predict(normalized_x_test)
```

In [165…
```
# Model Evaluation - Accuracy - for Train data

LR_mob_SM_custom.score(x_train_SM, y_train_SM)
```

Out[165… 0.7296396396396396

In [166…
```
# Classification Report - Train data

print(classification_report(y_train_SM, y_train_predict_SM_custom))
```

```
              precision    recall  f1-score   support

           0       0.73      0.72      0.73      5550
           1       0.73      0.74      0.73      5550

    accuracy                           0.73     11100
   macro avg       0.73      0.73      0.73     11100
weighted avg       0.73      0.73      0.73     11100
```

In [167…
```
# Classification Report - Test data

print(classification_report(y_test, y_test_predict_SM_custom))
```

```
              precision    recall  f1-score   support

           0       0.94      0.72      0.82      2380
           1       0.32      0.74      0.45       425

    accuracy                           0.72      2805
   macro avg       0.63      0.73      0.63      2805
```

```
weighted avg       0.85      0.72      0.76      2805
```

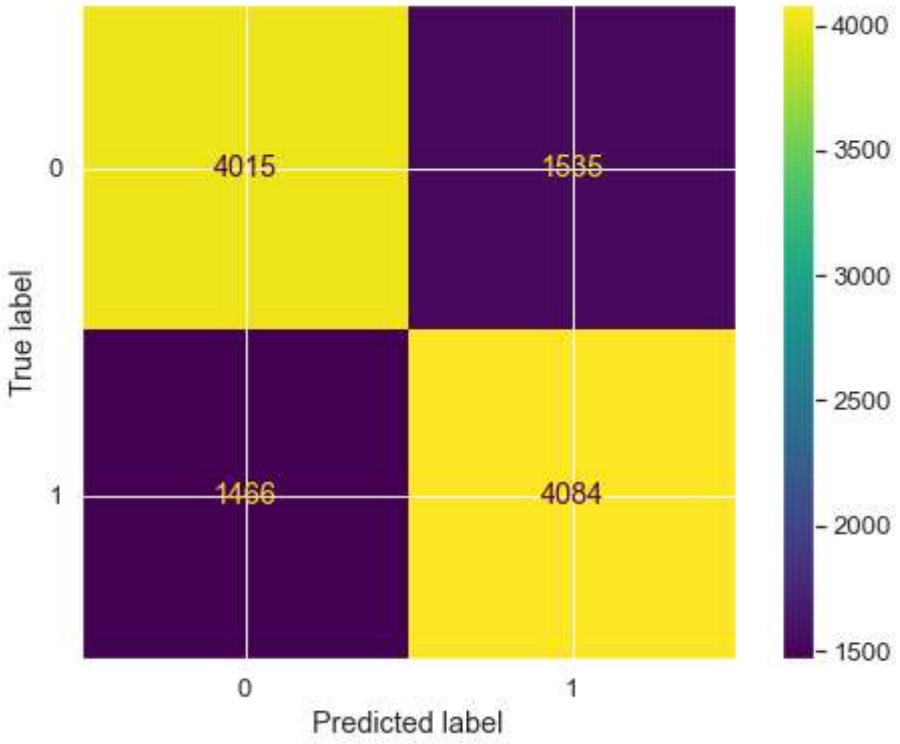
Confusion Matrix, AUC, ROC for the Model :-

In [168…

```
## Confusion matrix - Train data

plot_confusion_matrix(LR_mob_SM, x_train_SM, y_train_SM)
```

Out[168…

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x23650cda2b0>
```



So, True Positives are coming here is 472, that means 472 times our model has predicted correctly, that means it has predicted 1 as 1, and only 116 times it has predicted incorrectly.

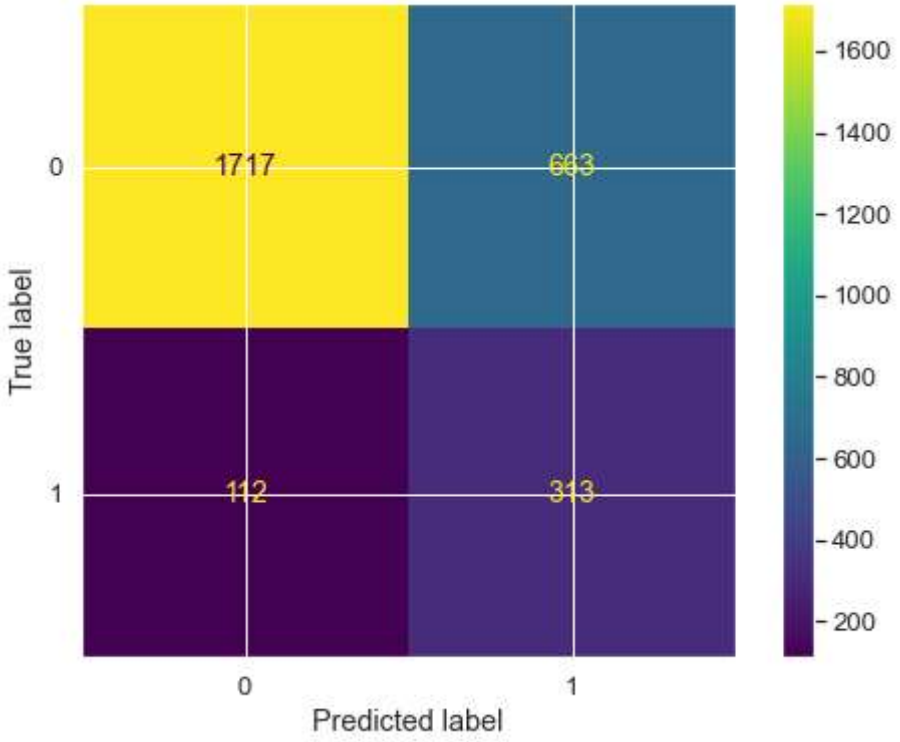
Similarly, True Negatives are 454 that means 454 times it has predicted 0 as 0 and only 116 times, the prediction was incorrect.

In [169…

```
## Confusion matrix - Test data

plot_confusion_matrix(LR_mob_SM, normalized_x_test, y_test)
```

Out[169…

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x23650d71fa0>
```



So, in test data, True Positives are coming here is 62, that means 62 times our model has predicted correctly, that means it has predicted 1 as 1, and only 27 times it has predicted incorrectly.

Similarly, True Negatives are 182 that means 182 times it has predicted 0 as 0 and only 62 times, the prediction was incorrect.

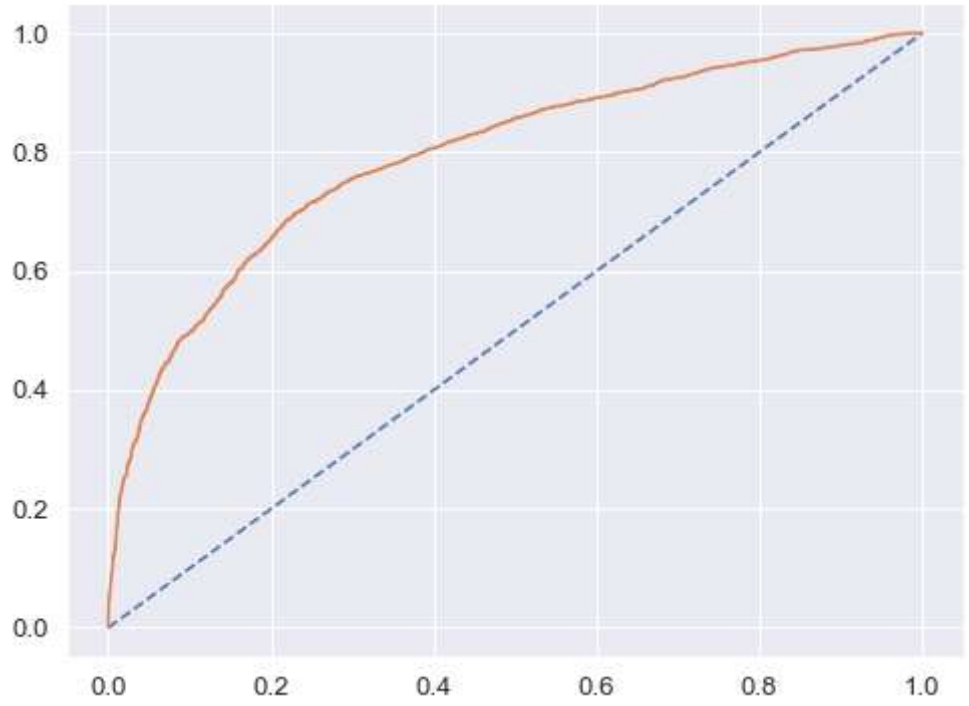
In [170…

```
# Predicting Probabilities

probs = LR_mob_SM.predict_proba(x_train_SM)
```

In [171…

```
# Keep probabilities for the positive outcome only

probs = probs[:, 1]

# Calculate Area Under Curve

auc = roc_auc_score(y_train_SM, probs)

print('AUC: %.3f' % auc)

# Calculate ROC curve

train_fpr, train_tpr, train_thresholds = roc_curve(y_train_SM, probs)
plt.rcParams["figure.figsize"] = (8,6)
plt.plot([0, 1], [0, 1], linestyle='--')


# Plot the roc curve for the model
plt.plot(train_fpr, train_tpr);
```

```
AUC: 0.794
```



In [172…
```
#Accuracy score - for Test data

acc_test = LR_mob_SM.score(normalized_x_test, y_test)

acc_test
```

Out[172…
```
0.7237076648841355
```

So in the above model, the area under the curve is 86% that means the overall model performance is good.

Also, the confusion matrix and classification report the LR_mob_SM Logistic Regression Model is giving the good results.

Using GridSearchCV for Logistic Regression :-

In [173…
```
grid = {'penalty':['l2','none'], 'solver':['sag','lbfgs'], 'tol':[0.0001,0.00001]}

LR_mob_grid = LogisticRegression(max_iter=10000, n_jobs=2)

grid_search = GridSearchCV(estimator=LR_mob_grid, param_grid = grid, cv=3, n_jobs=-1, s

grid_search.fit(x_train_SM, y_train_SM)
```

Out[173…
```
GridSearchCV(cv=3, estimator=LogisticRegression(max_iter=10000, n_jobs=2),
             n_jobs=-1,
             param_grid={'penalty': ['l2', 'none'], 'solver': ['sag', 'lbfgs'],
                         'tol': [0.0001, 1e-05]},
             scoring='f1')
```

In [174…
```
print('Best Parameters from GridSearch:',grid_search.best_params_)
```

```
print('\nBest estimator from GridSearch:',grid_search.best_estimator_)

best_LR_mob_model = grid_search.best_estimator_
```

```
Best Parameters from GridSearch: {'penalty': 'l2', 'solver': 'sag', 'tol': 0.0001}

Best estimator from GridSearch: LogisticRegression(max_iter=10000, n_jobs=2, solver='sa
g')
```

In [175…

```
# Prediction - Train data

y_train_grid_predict = best_LR_mob_model.predict(x_train_SM)
y_test_grid_predict = best_LR_mob_model.predict(normalized_x_test)
```

In [176…

```
# Getting the probabilities - Test data

y_test_grid_predict_prob = best_LR_mob_model.predict_proba(normalized_x_test)

pd.DataFrame(y_test_grid_predict_prob).head()
```

Out[176…

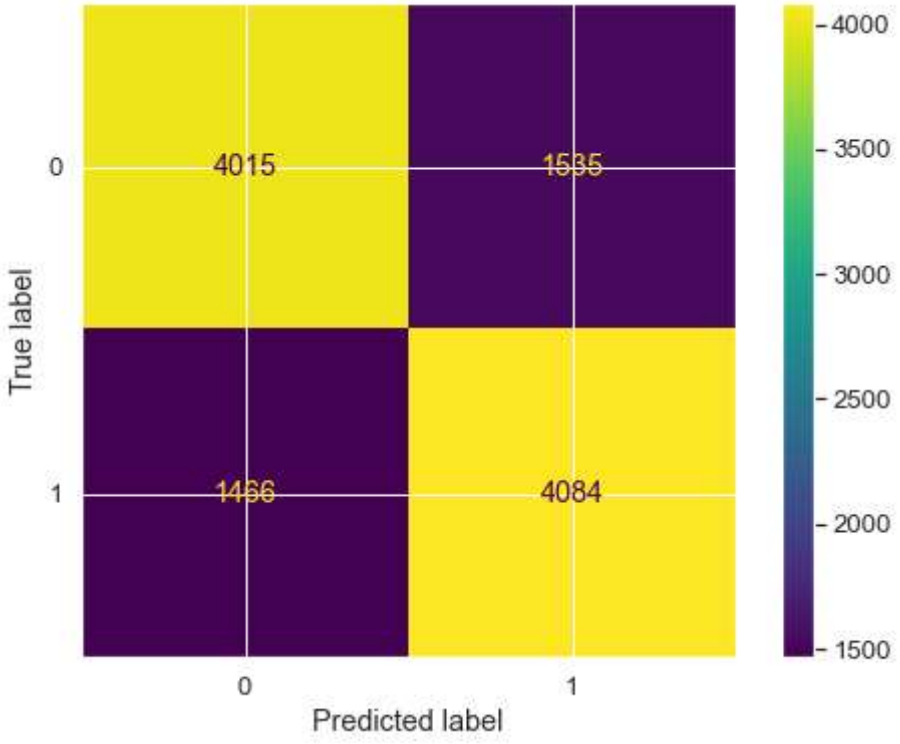
	0	1
0	0.54	0.46
1	0.15	0.85
2	0.57	0.43
3	0.80	0.20
4	0.60	0.40

In [177…

```
# Confusion Matrix - Train data

plot_confusion_matrix(best_LR_mob_model, x_train_SM, y_train_SM)
```

Out[177…

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x23650d7e670>
```



In [178…

```
# Classification Report - Train data

print(classification_report(y_train_SM, y_train_grid_predict))
```

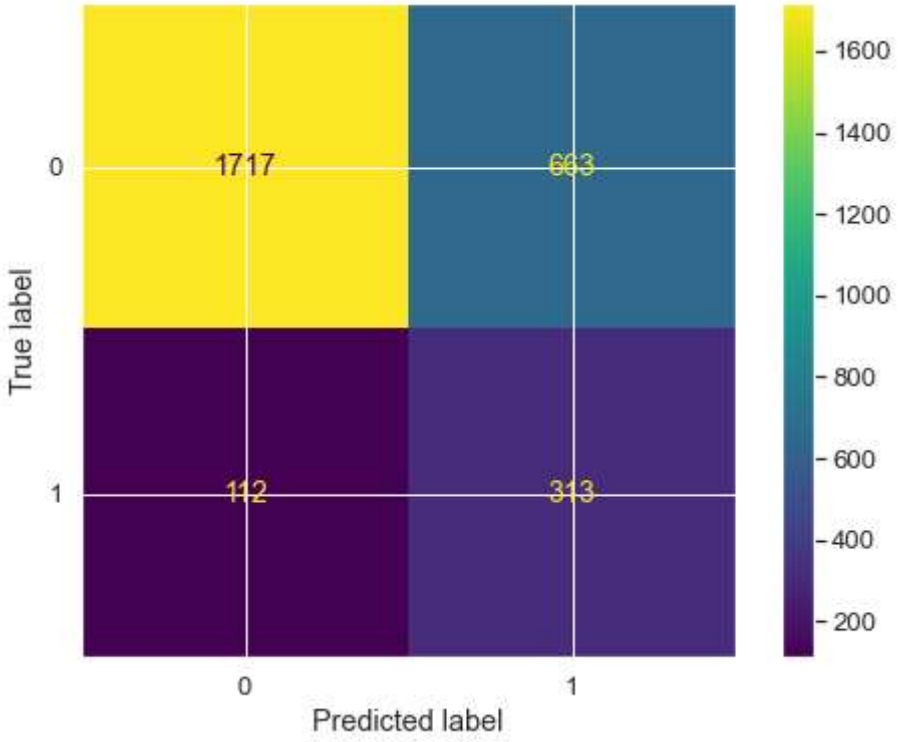
```
              precision    recall  f1-score   support

           0       0.73      0.72      0.73      5550
           1       0.73      0.74      0.73      5550

    accuracy                           0.73     11100
   macro avg       0.73      0.73      0.73     11100
weighted avg       0.73      0.73      0.73     11100
```

In [179…

```
# Confusion Matrix - Test data

plot_confusion_matrix(best_LR_mob_model, normalized_x_test, y_test)
```

Out[179…

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x23650bf02e0>
```



In [180…
```
# Classification Report - Test data

print(classification_report(y_test, y_test_grid_predict))
```

```
              precision    recall  f1-score   support

           0       0.94      0.72      0.82      2380
           1       0.32      0.74      0.45       425

    accuracy                           0.72      2805
   macro avg       0.63      0.73      0.63      2805
weighted avg       0.85      0.72      0.76      2805
```

KNN Model (k-nearest neighbors)

We will be building KNN model on re-sampled train sets.

In [181…
```
# Importing KNeighboursClassifier

KNN_mob_model = KNeighborsClassifier()

KNN_mob_model.fit(x_train_SM, y_train_SM)
```

Out[181…
```
KNeighborsClassifier()
```

In [182…
```
#let's predict the model on train set and check the model score

predicted_labels = KNN_mob_model.predict(x_train_SM)

KNN_mob_model_score = KNN_mob_model.score(x_train_SM, y_train_SM)
```

```
KNN_mob_model_score
```

Out[182… 0.9808108108108108

Classification Report & Confusion Matrix :-

In [183…

```
print('Confusion Matrix\n')

print(metrics.confusion_matrix(y_train_SM, predicted_labels))

print('Classification Report\n')

print(metrics.classification_report(y_train_SM, predicted_labels))
```

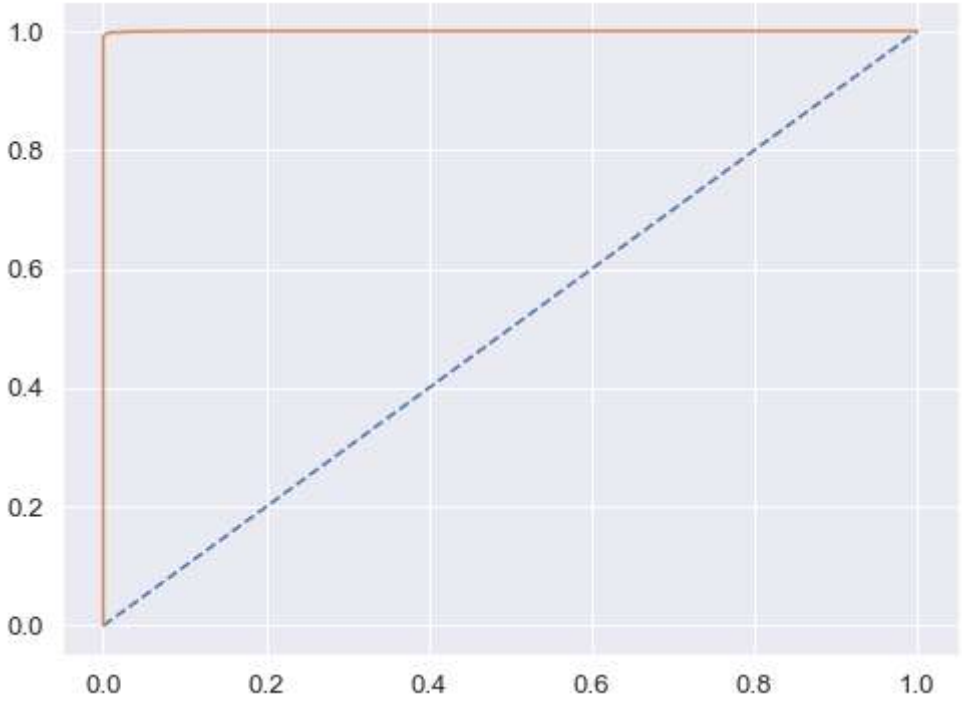
```
Confusion Matrix

[[5342  208]
 [   5 5545]]
Classification Report

              precision    recall  f1-score   support

           0       1.00      0.96      0.98      5550
           1       0.96      1.00      0.98      5550

    accuracy                           0.98     11100
   macro avg       0.98      0.98      0.98     11100
weighted avg       0.98      0.98      0.98     11100
```

AUC ROC Curve on Train data :-

In [184…

```
probs_train = KNN_mob_model.predict_proba(x_train_SM)

probs_train = probs_train[:, 1]

auc=roc_auc_score(y_train_SM,probs_train)

print("Area Under Curve: %.3f " % auc)
train_fpr ,train_tpr, train_thresholds= roc_curve(y_train_SM, probs_train)
plt.plot([0,1],[0,1],linestyle='--')
plt.plot(train_fpr,train_tpr);
```

```
Area Under Curve: 1.000
```



Naive Bayes Model

In [185…
```
# Fit the model into training data

NB_mob_model = GaussianNB()

NB_mob_model.fit(x_train_SM, y_train_SM)
```

Out[185…
```
GaussianNB()
```

In [186…
```
NB_train_predict = NB_mob_model.predict(x_train_SM)

print("Model Accuracy: {0:.2f}".format(metrics.accuracy_score(y_train_SM, NB_train_pred
```

```
Model Accuracy: 0.69
```

In [187…
```
# Prediction - on Test data

NB_test_predict = NB_mob_model.predict(normalized_x_test)
```

In [188…
```
print("Model Accuracy: {0:.2f}".format(metrics.accuracy_score(y_test, NB_test_predict))
```

```
Model Accuracy: 0.66
```

In [234…
```
# Confusion Matrix

print("Confusion Matrix:\n")

CM = metrics.confusion_matrix(y_test, NB_test_predict, labels=[1, 0])

df_CM = pd.DataFrame(CM, index = [i for i in ["1","0"]],
```

```
                    columns = [i for i in ["Predict 1","Predict 0"]])

# Plotting the matrix
plt.figure(figsize = (8,6))
sns.heatmap(df_CM, annot=True)
```

```
Confusion Matrix:
```

Out[234…

```
<AxesSubplot:>
```

In [190…

```
# Plotting the matrix
plt.figure(figsize = (8,6))
sns.heatmap(df_CM, annot=True)

print("Classification Report")
print(metrics.classification_report(y_test, NB_test_predict, labels=[1, 0]))
```

```
Classification Report
              precision    recall  f1-score   support

           1       0.27      0.73      0.40       425
           0       0.93      0.65      0.77      2380

    accuracy                           0.66      2805
   macro avg       0.60      0.69      0.58      2805
weighted avg       0.83      0.66      0.71      2805
```

2. Model Tuning

1. Ensemble modeling, wherever applicable.
2. Any other model tuning measures(if applicable).
3. Interpretation of the most optimum model and its implication on the business.

Random Forest

In [191…
```
rfcl_mob = RandomForestClassifier(n_estimators=301, oob_score = True, max_depth = 10, m

rfcl_mob = rfcl_mob.fit(x_train_SM, y_train_SM)
```

n_estimators = number of trees, that we want to build within the RandomForestClassifier.

Out of bag (OOB) score is a way of validating the Random forest model. OOB score is computed as the number of correctly predicted rows from the out of bag sample.

max_features will be used to select randomly features from the available number of features.

In [192…
```
rfcl_mob
```

Out[192…
```
RandomForestClassifier(max_depth=10, max_features=5, min_samples_leaf=50,
                       min_samples_split=110, n_estimators=301, oob_score=True)
```

In [193…
```
rfcl_mob.oob_score_
```

Out[193…
```
0.8774774774774775
```

In [194…
```
#Checking the error rate in %:
```

```
print('Error_rate(in %) :', 1- (rfcl_mob.oob_score_)) #OOB Error is the number of wrong
```

```
Error_rate(in %) : 0.12252252252252249
```

For the above specified grid parameters, we got the rfcl_mob model with oob score around 83% and error rate 17%, which shows the model is good to go for further evaluation.

Getting the Feature Importances :-

In [195…

```
print (pd.DataFrame(rfcl_mob.feature_importances_, columns = ["Imp"],
                    index = x_train_SM.columns).sort_values('Imp',ascending=False))
```

```
                              Imp
following_CP_Yes             0.19
TL_rec_on_OS_checkin         0.17
Adult_flag                   0.14
YA_view_on_CP                0.09
week_since_last_OS_checkin   0.08
DA_mins_spend_on_CP          0.08
TL_done_on_OS_checkin        0.07
travelling_network_rating    0.05
YA_OS_checkin                0.05
members_in_fam               0.03
YA_comment_on_CP             0.03
working_flag_Yes             0.00
pref_loc_type_Entertainment  0.00
pref_loc_type_Medical        0.00
pref_loc_type_Financial      0.00
pref_loc_type_Big Cities     0.00
pref_loc_type_Other          0.00
```

Let's Now use Ensemble techniques : Bagging, Ada Boosting, GradientBoost :-

In [196…

```
#Let's first look by use Bagging

#we will be using random forest as base_estimator

bgcl_mob = BaggingClassifier(base_estimator=rfcl_mob, n_estimators=50,random_state=1)
```

In [197…

```
# Fit the model into training data
bgcl_mob = bgcl_mob.fit(x_train_SM, y_train_SM)
```

In [198…

```
# Let's predict on test set

y_test_predict = bgcl_mob.predict(normalized_x_test)
```

In [199…

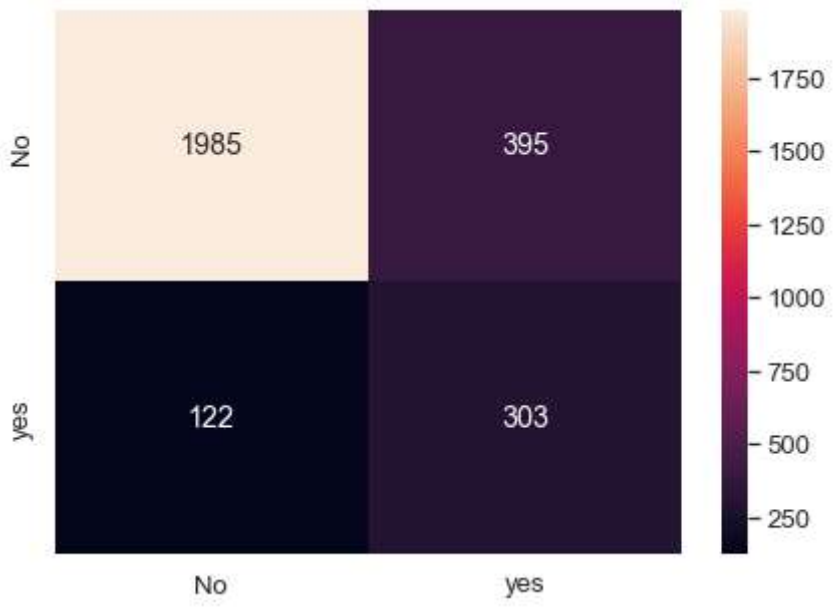
```
print(bgcl_mob.score(normalized_x_test , y_test))
```

```
0.8156862745098039
```

In [200…

```
cm=metrics.confusion_matrix(y_test, y_test_predict,labels=[0, 1])

df_cm = pd.DataFrame(cm, index = [i for i in ["No","yes"]],
                     columns = [i for i in ["No","yes"]])

plt.figure(figsize = (7,5))
sns.heatmap(df_cm, annot=True ,fmt='g')
```

Out[200… <AxesSubplot:>



Ada Boosting

In [201…

```
# Defining the ada boosting classifier

abcl_mob = AdaBoostClassifier(n_estimators=10, random_state=1)
```

In [202…

```
#fit the model to the train data

abcl_mob = abcl_mob.fit(x_train_SM, y_train_SM)
```

In [203…

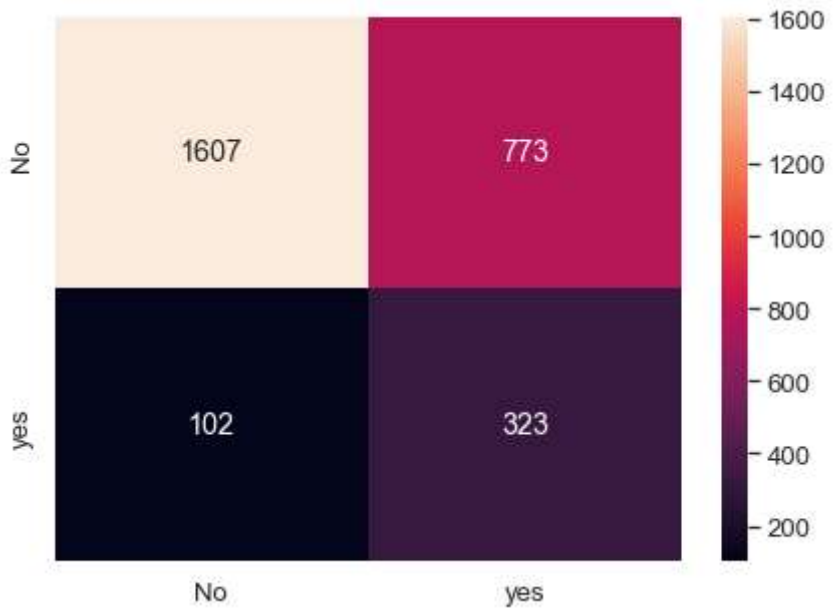
```
# Prediction on - Test data

y_predict_1 = abcl_mob.predict(normalized_x_test)

print(abcl_mob.score(normalized_x_test , y_test))
```

```
0.6880570409982175
```

In [204…

```
cm = metrics.confusion_matrix(y_test, y_predict_1,labels=[0, 1])

df_cm_1 = pd.DataFrame(cm, index = [i for i in ["No","yes"]],
                     columns = [i for i in ["No","yes"]])
plt.figure(figsize = (7,5))
sns.heatmap(df_cm_1, annot=True ,fmt='g')
```

Out[204… <AxesSubplot:>



Gradient Boost

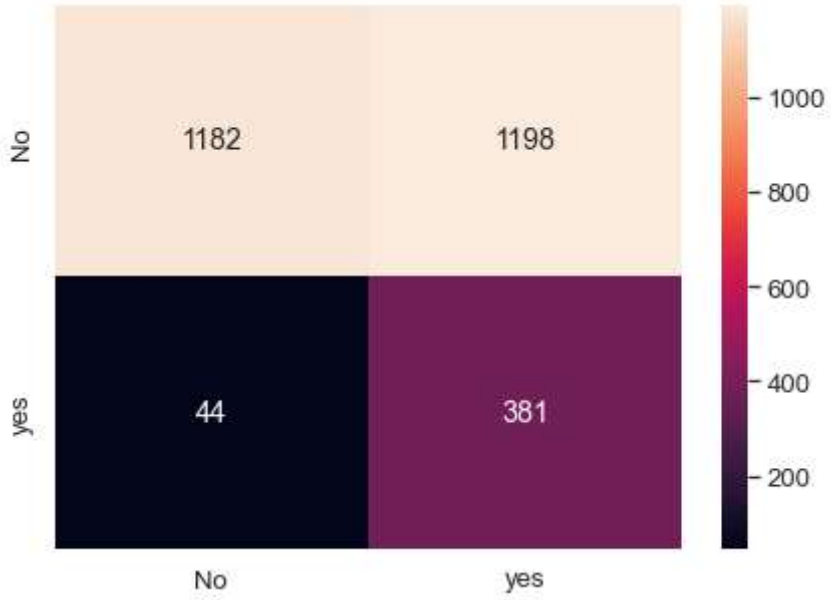
In [205…
```
gbcl_mob = GradientBoostingClassifier(n_estimators = 50,random_state=1)

# Fit the model on Train data
gbcl_mob = gbcl_mob.fit(x_train_SM, y_train_SM)
```

In [206…
```
# Prediction - on Test data

y_predict_2 = gbcl_mob.predict(normalized_x_test)
print(gbcl_mob.score(normalized_x_test, y_test))
```

```
0.5572192513368984
```

In [207…
```
cm = metrics.confusion_matrix(y_test, y_predict_2,labels=[0, 1])

df_cm_2 = pd.DataFrame(cm, index = [i for i in ["No","yes"]],
                  columns = [i for i in ["No","yes"]])
plt.figure(figsize = (7,5))
sns.heatmap(df_cm_2, annot=True ,fmt='g')
```

Out[207… <AxesSubplot:>



In [ ]: